

"Own the service boundary before you own the code — size, data, and failure are design choices."

SUPPLEMENTARY · MODULE 1

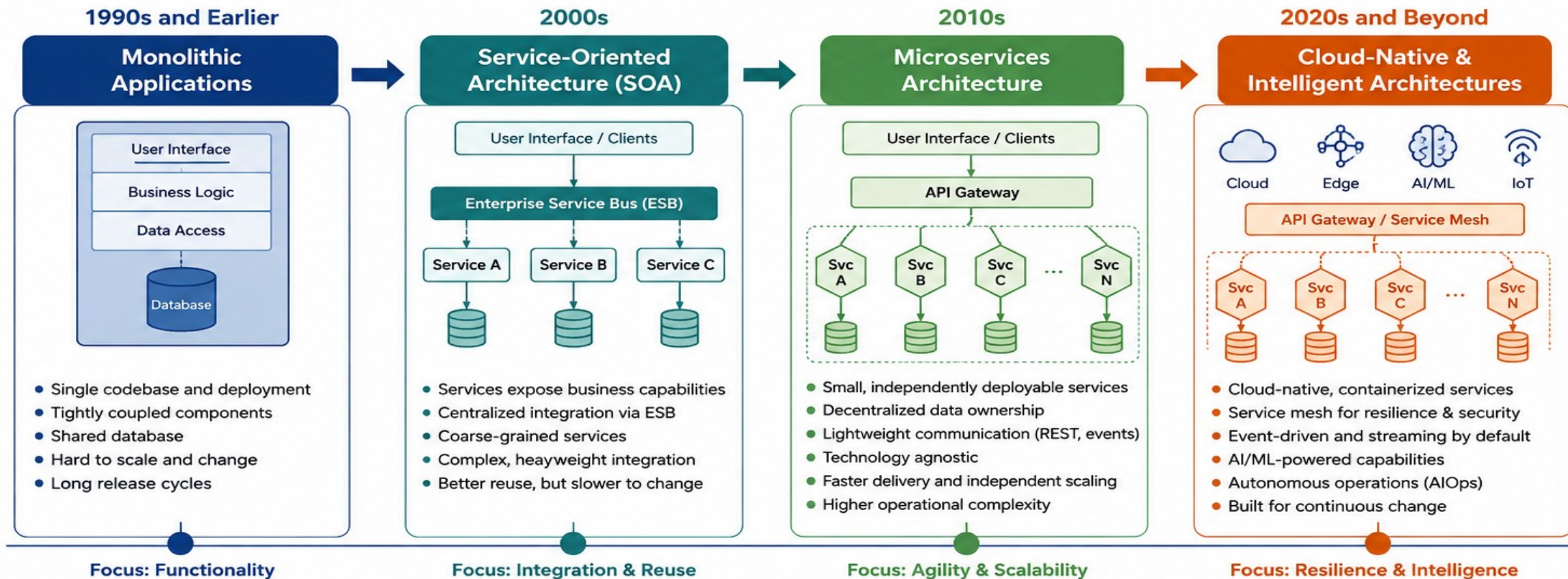
Microservices Fundamentals and Cloud-Native

Turn the banking estate into a shared language: bounded contexts, twelve-factor habits, OpenShift as the runtime, and the trade-offs that keep a service independently deployable.



Evolution of Enterprise Application Architecture

Enterprise application architecture has evolved to meet changing business needs for speed, scalability, and resilience.



Key Takeaway

Architectures have evolved from monolithic systems to intelligent, cloud-native platforms to deliver business value faster, operate at scale, and adapt to constant change.



Business-Driven Design



Technology Enablers



Speed & Adaptability



Resilience & Security

TRADITIONAL MONOLITHIC ARCHITECTURE

A monolith deploys all layers as one tightly coupled unit sharing a single codebase and database.



ARCHITECTURE LAYERS

- **Users/Clients:** Web, Mobile, Third-Party Systems
- Presentation Layer (UI / Controllers)
- Business Logic Layer (Services)
- Data Access Layer (Repositories / DAOs)
- Persistence Layer (Database Access) to a Single Database
- Deployed as a Single Artifact (WAR/JAR) to a Single Unit



HOW IT WORKS / KEY LIMITATIONS

- All features and components are part of one codebase
- Deployed as a single application to an application server
- All modules share the same process and memory space
- Tight coupling makes changes riskier and harder
- Difficult to scale specific parts independently; technology stack must be uniform



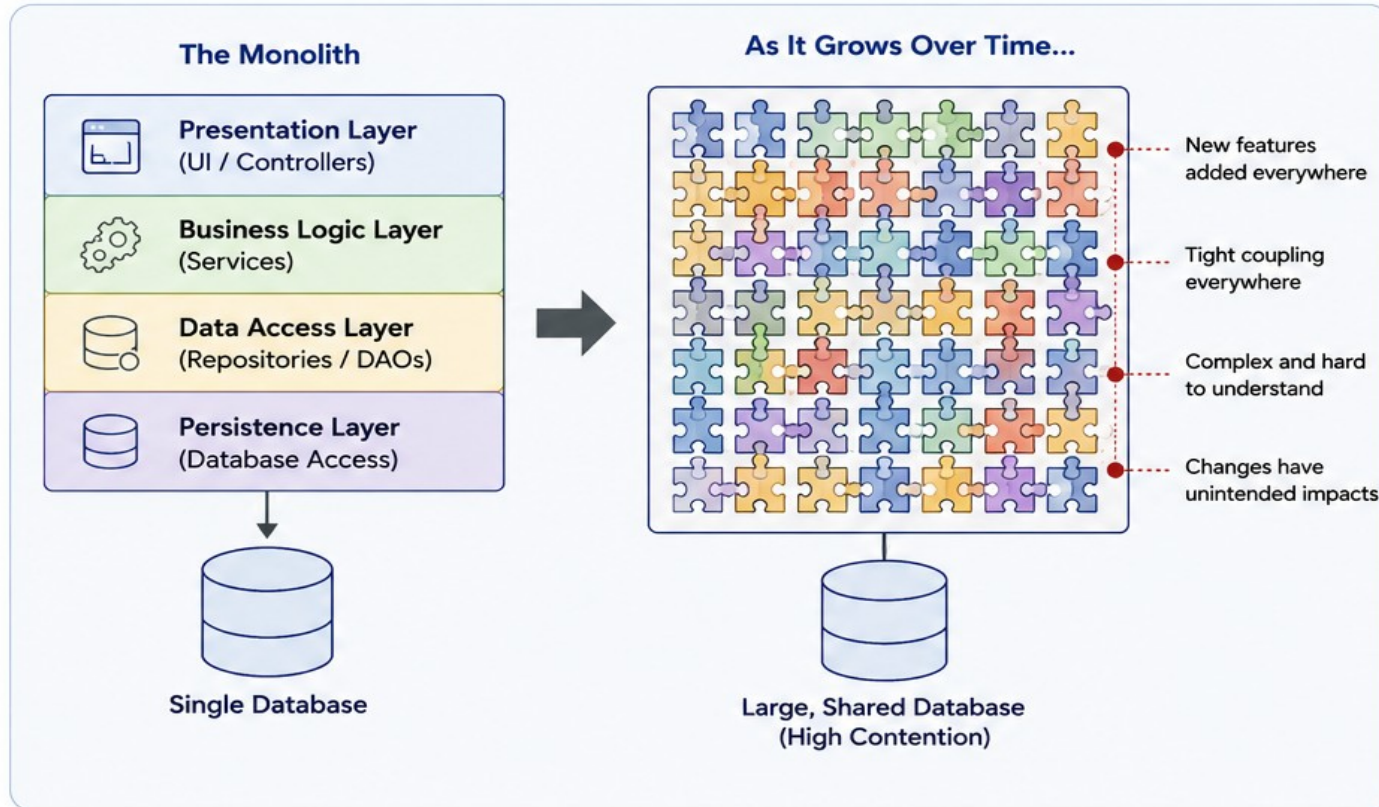
TYPICAL COMPONENTS / COMMON USE CASES

- **Typical components:** user interface, business logic, data access and persistence, shared libraries
- **Common use cases:** simple or small applications, early-stage startups, internal tools
- Applications with low scale or change frequency

KEY TAKEAWAY Monoliths are simple to start with, but become complex and costly to evolve as the application grows.

Problems with Growing Monolithic Applications

As a monolithic application grows, it becomes harder to change, scale, and deliver value quickly.



Key Problems



Slower Development

Code base becomes large and complex. Changes take longer and require more testing.



Tight Coupling

Components are highly dependent on each other. Small changes can break unrelated functionality.



Harder to Test

Large, interdependent codebase makes unit testing difficult and slow.



Poor Scalability

Cannot scale parts of the application independently. Entire application must scale.



Database Bottlenecks

Single database becomes a hotspot. Contention and locks impact performance.



Difficult Deployments

Must deploy the entire application for any change. Higher risk and longer release cycles.



Team Productivity Drops

Teams step on each other's code. Hard to work in parallel.



Higher Operational Costs

More infrastructure required to scale the whole application. Inefficient resource usage.

Business Impact



Slower time to market



Higher defect rates



Reduced agility



Lower team morale



Higher total cost of ownership



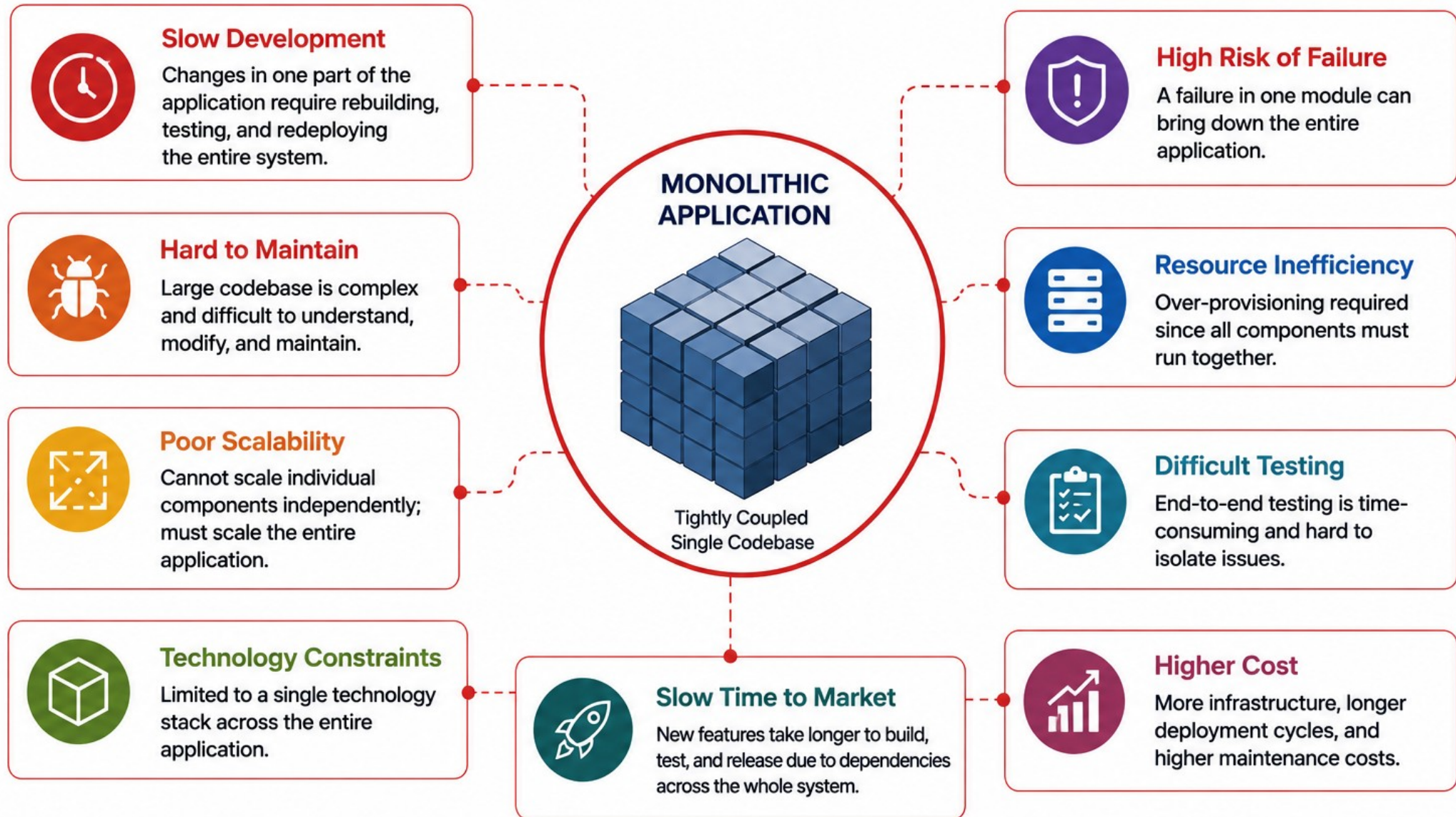
The Takeaway

Monoliths can work for simple applications, but as they grow, they become expensive, risky, and slow. This is why organizations evolve toward modular and microservices architectures.



Growing monoliths increase complexity and cost while reducing agility and reliability.

Problems with Monolithic Architectures



MODULAR MONOLITHS EXPLAINED

A modular monolith organizes one codebase into well-defined modules, easing the path to microservices.



WHAT IS A MODULAR MONOLITH? - KEY CHARACTERISTICS

- Single codebase and deployment unit
- Modular structure with clear boundaries
- Encapsulation and information hiding within modules
- Defined contracts between modules
- Shared infrastructure (database, UI, tooling)
- Enables parallel development within a single application



BENEFITS

- **Better Organization:** code organized by business capability, not technical layers
- **Improved Team Productivity:** teams work on different modules with minimal interference
- **Lower Risk of Change:** encapsulation reduces unintended impacts across the application
- **Easier Testing:** modules can be tested in isolation
- **A step toward microservices:** modules with clear boundaries can be extracted over time



WHEN TO USE A MODULAR MONOLITH

- The application is growing but still fits in a single database
- Multiple teams need to collaborate on the same application
- You want to improve structure before considering microservices
- You need to reduce risk before making architectural changes

KEY TAKEAWAY Think of a modular monolith as building with smart bricks today, so you can build with microservices tomorrow.

WHAT IS A MICROSERVICE

A microservice is a small, independent service owning one business capability, connected via APIs.



MICROSERVICES IN SIMPLE TERMS

- Think of microservices as a collection of specialized, autonomous services
- Each service owns a specific business function and communicates via APIs
- Small and focused
- Independent codebase
- Own data store
- Exposed via APIs
- Built, deployed, and scaled independently



MICROSERVICES VS. MONOLITH

ITEM	MONOLITH	MICROSERVICES
Codebase	single codebase	multiple small services
Coupling	tightly coupled components	loosely coupled
Database	shared database	each service owns its data
Scaling	hard to scale	scale services independently
Deployments	large deployments	small, frequent deployments
Technology	technology lock-in	technology flexibility

KEY TAKEAWAY Microservices break a large application into a suite of small, independent services that collaborate to deliver business value.

WHAT IS A MICROSERVICE

A microservice is a small, independent service owning one business capability, connected via APIs.



HOW MICROSERVICES WORK TOGETHER

- Clients (web, mobile, browser) connect through a single API Gateway
- Gateway routes requests to User, Order, Payment, Notification, and Inventory Services
- Each service owns its own dedicated database
- Services are organized around business capabilities

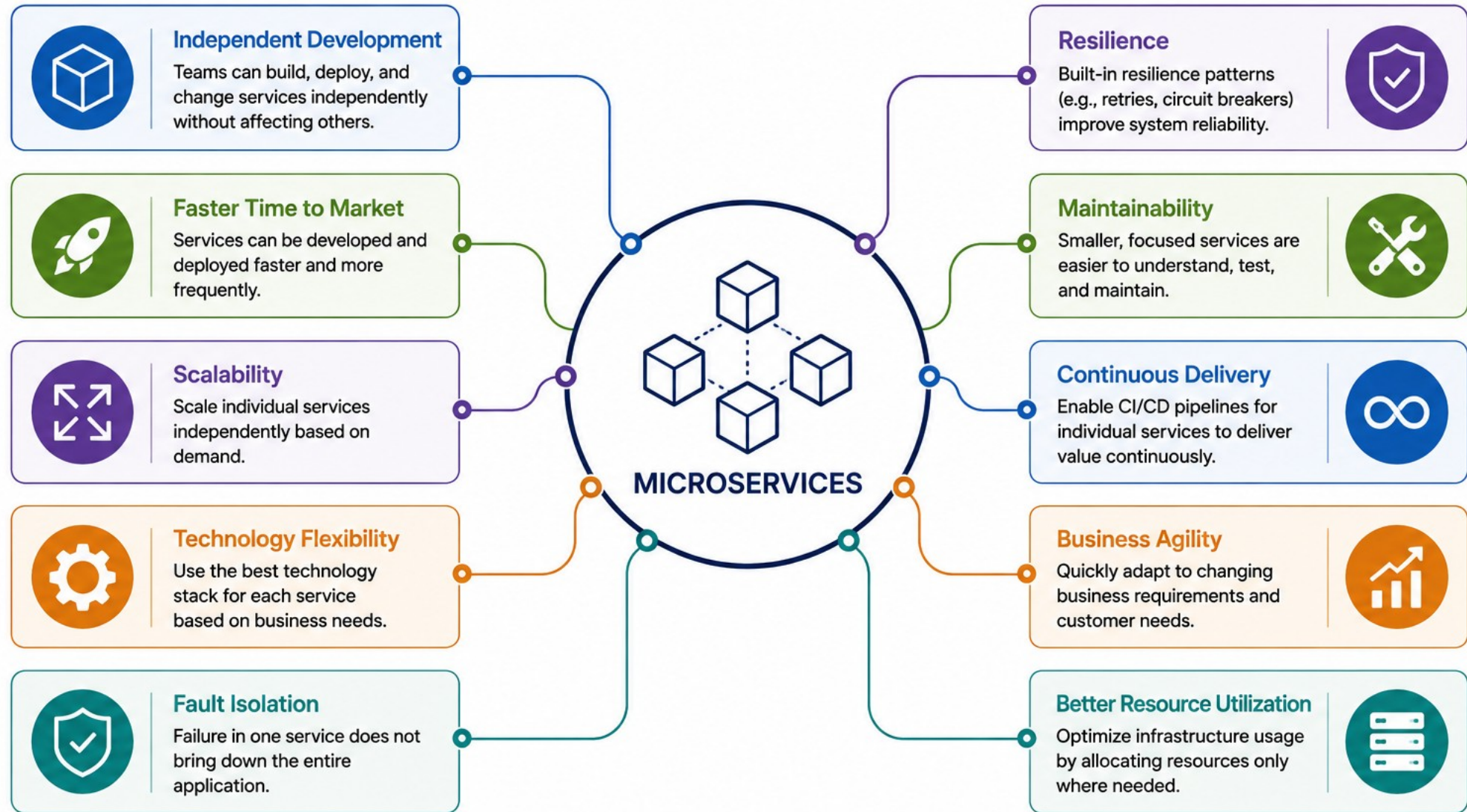


WHY MICROSERVICES?

- **Agility:** deliver features faster with small, independent teams
- **Scalability:** scale only the services that need more capacity
- **Resilience:** isolated failures improve system stability and availability
- **Cost Efficiency:** optimize resource usage and reduce infrastructure costs
- **Technology Freedom:** use the best tool or database for each specific service
- **Team Empowerment:** small, cross-functional teams own services end-to-end

KEY TAKEAWAY Microservices break a large application into a suite of small, independent services that collaborate to deliver business value.

Why Microservices?



Microservices vs Monoliths

MICROSERVICES	VS	MONOLITHS
 Architecture Application is broken down into small, independent services that communicate over APIs.	 Architecture All functionality is built and deployed as a single, unified application.	 Architecture All functionality is built and deployed as a single, unified application.
 Development Teams can develop, deploy, and release services independently and in parallel.	 Development All changes follow a single codebase and release cycle.	 Development All changes follow a single codebase and release cycle.
 Deployment Services are deployed independently without affecting others.	 Deployment The entire application must be deployed even for small changes.	 Deployment The entire application must be deployed even for small changes.
 Scalability Scale individual services based on demand.	 Scalability Scale the entire application even if only one part needs it.	 Scalability Scale the entire application even if only one part needs it.
 Fault Isolation Failure in one service does not affect the entire system.	 Fault Isolation Failure in one module can bring down the whole application.	 Fault Isolation Failure in one module can bring down the whole application.
 Technology Different services can use the best technology for their needs.	 Technology Limited to a single technology stack across the entire application.	 Technology Limited to a single technology stack across the entire application.
 Time to Market Faster delivery of new features due to independent development.	 Time to Market Slower delivery due to dependencies and longer release cycles.	 Time to Market Slower delivery due to dependencies and longer release cycles.
 Maintainability Smaller codebases are easier to understand, test, and maintain.	 Maintainability Large codebase is complex and harder to maintain.	 Maintainability Large codebase is complex and harder to maintain.
 Cost Efficient resource usage; pay for what you use.	 Cost Higher infrastructure cost due to over-provisioning.	 Cost Higher infrastructure cost due to over-provisioning.

When a Monolith Is the Better Choice

Monoliths are simple, cost-effective, and fast to deliver in the right situations.

Choose a Monolith When...



1. Early-Stage Product

You are validating an idea or building a Minimum Viable Product (MVP). Speed and simplicity matter most.



2. Small Team

A single team can build, deploy, and manage the application efficiently without complex processes.



3. Simple Domain

The business domain is straightforward with limited complexity and minimal evolving requirements.



4. Low Scale Needs

Expected traffic and data volumes are modest and can be handled by a single deployable.



5. Limited Budget

You need to minimize infrastructure, tooling, and operational costs.



6. Fast Time to Market

You need to deliver features quickly without the overhead of distributed systems.



7. Tight Feedback Loop

A monolith enables easier end-to-end testing, debugging, and rapid iteration.



8. Low Operational Maturity

Your team is still building DevOps, monitoring, and incident-response capabilities.



Why a Monolith Can Be the Smart Choice



Reduced complexity in development, deployment, and operations.



Lower infrastructure and tooling costs.



Easier to secure and manage as a single unit.



Simpler monitoring, logging, and troubleshooting.



Better for teams still building their engineering practices.



Keep in Mind

A monolith can become a liability if the domain grows in complexity, the team scales, or you need independent scaling and frequent deployments.

Signs You May Outgrow a Monolith



Rapidly growing feature set



Multiple teams working in parallel



Different data and scaling needs



Slow deployments and high change risk



Technology and architectural constraints



Bottom Line

Start with a monolith when simplicity, speed, and cost matter most. Evolve to microservices when complexity, scale, and team structure demand it.



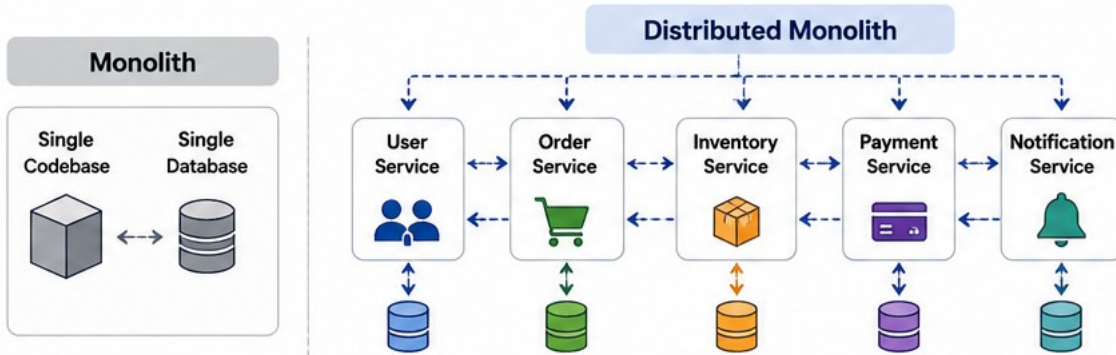
The Distributed Monolith

Looks like microservices. Acts like a monolith. Loses the benefits of both.



What Is a Distributed Monolith?

A system decomposed into multiple services but still tightly coupled in code, data, and deployment, preventing independent evolution and scaling.



- ❌ Tight synchronous calls
- ❌ Shared database or schema
- ❌ Shared business logic or libraries
- ❌ Coordinated deployments
- ❌ Changes require multiple teams



Why It's a Problem



Tight Coupling

A change in one service impacts many others.



Cannot Scale Independently

All services must scale together even if only one is under load.



Slow Delivery

Frequent coordination leads to longer release cycles.



High Failure Impact

Failures cascade across multiple services.



Operational Complexity

More services, but still monolithic thinking.



Common Signs



Frequent Synchronous Calls

Most requests are synchronous and chained across services.



Shared Database or Schema

Services read/write the same database or schema.



Shared Code and Libraries

Common libraries contain business logic and rules.



Coordinated Deployments

Services must be deployed together or in a specific order.



Centralized Ownership

One team or group owns most services and decisions.



How to Avoid It

- ✓ Design around business capabilities and bounded contexts.
- ✓ Use asynchronous communication and events where possible.
- ✓ Own your data. Avoid shared databases and schemas.
- ✓ Keep services small, cohesive, and independently deployable.
- ✓ Decentralize ownership and empower teams.



Remember

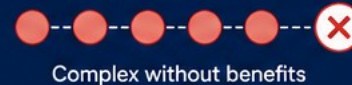
Microservices is about **INDEPENDENCE** not just **DISTRIBUTION**.



Key Takeaway

A distributed monolith gives you the complexity of microservices without the benefits. Focus on loose coupling, data ownership, and independent deployment to realize true agility and scale.

Distributed Monolith



Proper Microservices



WHEN MICROSERVICES ARE APPROPRIATE

Use microservices when complexity, scale, and business needs justify the added complexity.



MICROSERVICES ARE APPROPRIATE WHEN YOU NEED TO...

- 1. Scale Independently:** different parts of your system have different scaling requirements
- 2. Manage Complex Domains:** large complex business domain broken into loosely coupled bounded contexts
- 3. Multiple Teams and Ownership:** multiple teams need to own, build, and deploy services independently
- 4. Technology Diversity:** pick the language, framework, or data store each workload actually needs
- 5. Improve Resilience:** isolate failures so one struggling service cannot take the whole platform down
- 6. Faster Feature Delivery:** small, independently deployable units release without a system-wide cycle
- 7. Integration and Ecosystem:** expose capabilities as APIs and events for partners and internal consumers
- 8. Data Ownership:** each service owns its schema and lifecycle, ending contention over one shared database

KEY TAKEAWAY Choose microservices when the business and technical benefits outweigh the added operational and architectural complexity.

WHEN MICROSERVICES ARE APPROPRIATE

Use microservices when complexity, scale, and business needs justify the added complexity.



WHAT YOU GAIN

- **Independent Scalability:** scale only what you need, when you need it
- **Faster Innovation:** teams move faster without being blocked by other teams
- **Better Fault Isolation:** failures are contained within a service, improving resilience
- **Technology Freedom:** choose the best tools and frameworks for each service



INDICATORS YOU MAY NEED MICROSERVICES

- Large, growing organization
- Frequent organizational changes
- Bottlenecks in the monolith
- Different data security or privacy needs
- Need for cloud-native elasticity
- Monolith becomes hard to change and test
- **Bottom Line:** choose microservices when the business and technical benefits outweigh the overhead

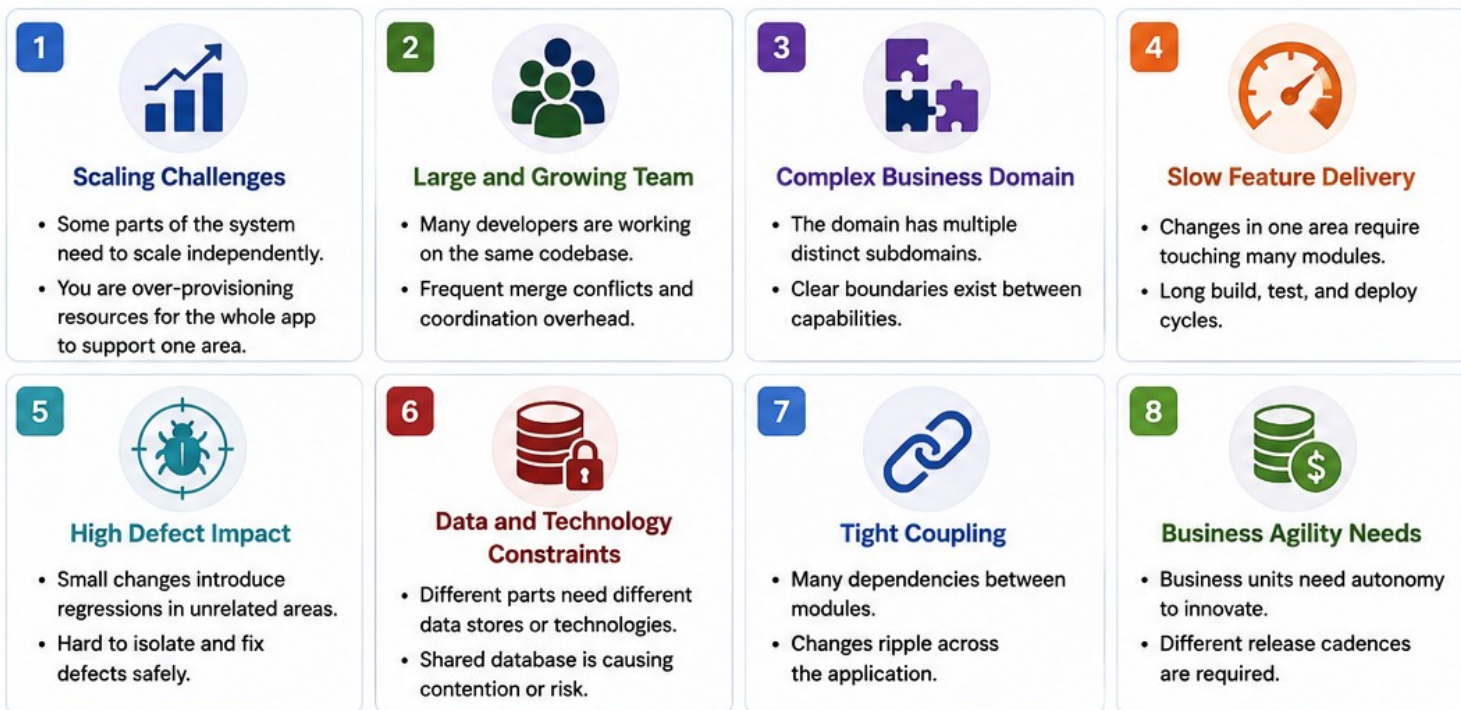
KEY TAKEAWAY Choose microservices when the business and technical benefits outweigh the added operational and architectural complexity.

Indicators for Decomposing an Application

Use these signals to decide when and where to break a monolith into microservices.



Key Indicators Your Application May Need to Be Decomposed



How to Use These Indicators



1. Assess

Evaluate your application against these indicators.



2. Prioritize

Focus on the areas with the strongest signals.



3. Identify Boundaries

Define clear bounded contexts and service candidates.



4. Plan Incrementally

Start small. Decompose one capability at a time.



5. Validate and Iterate

Continuously validate benefits and adjust your roadmap.

The Decomposition Spectrum



Be Cautious

Decomposing too early adds operational complexity. Use these indicators to make data-driven decisions, not assumptions.



Key Takeaway

Decompose when the pain of staying monolithic is greater than the cost of going distributed. Aim for business-aligned, loosely coupled, independently deployable services.



Common Microservices Misconceptions

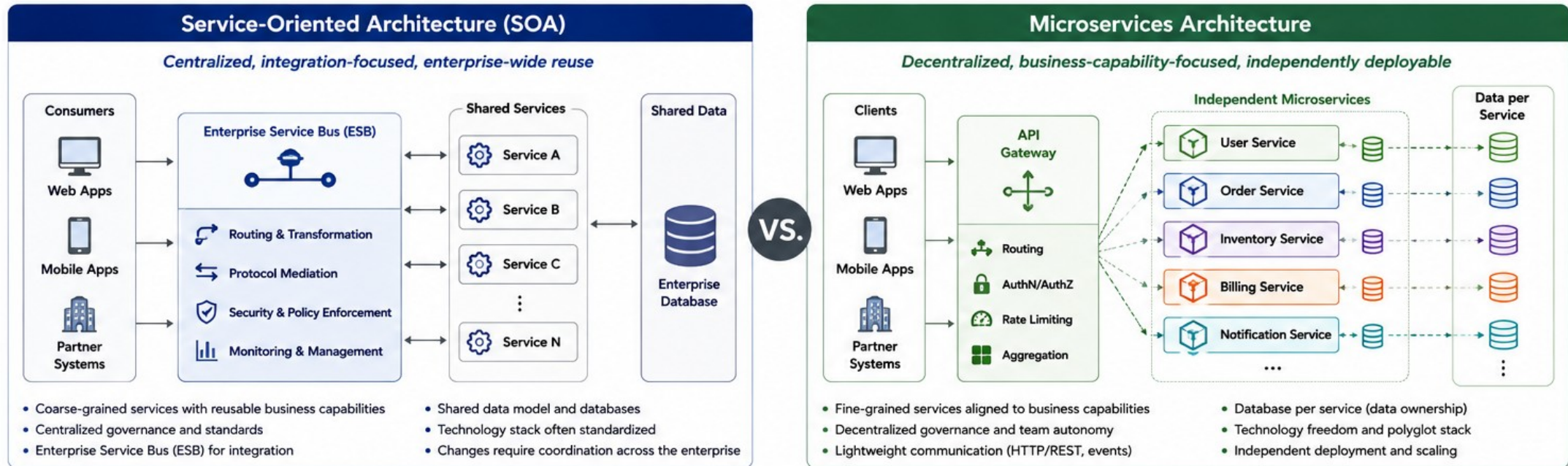
Don't be fooled by the hype. Microservices are a means to an end, not an end in themselves.

❌ MISCONCEPTION		✅ REALITY		💡 What to Remember
1	 Microservices are always faster and more scalable.	→	 Not automatically. Poorly designed microservices can be slower and harder to scale than a well-designed monolith.	 Start with business value. Solve business problems, not technology problems.
2	 Microservices are just small versions of a monolith.	→	 They are domain-driven. Services should be aligned to business capabilities with clear boundaries and ownership.	 Align with capabilities. Design around business domains and team ownership.
3	 Breaking into microservices is easy—just split the code.	→	 It's a complex journey. It requires careful planning, refactoring, data strategy, testing, and organizational readiness.	 Optimize for the whole system. Consider reliability, operability, and user experience.
4	 Each microservice needs its own database.	→	 Not always necessary. Database-per-service is common, but evaluate needs, consistency requirements, and operational cost.	 Measure and iterate. Use data to validate that microservices are delivering real benefits.
5	 Microservices mean moving everything to the cloud.	→	 Deployment model is a choice. Microservices can run on-prem, in the cloud, or hybrid. Focus on the architecture, not the cloud.	 Danger Signs • Creating a service for every feature • Duplicate data and logic across services • Overly chatty service calls • Lack of clear ownership • No automated testing or observability
6	 We need microservices to be DevOps.	→	 DevOps is about culture and practices. You can do DevOps with a monolith. Microservices help, but are not a prerequisite.	
7	 Microservices will reduce costs.	→	 They can increase costs. More services mean more infrastructure, tooling, monitoring, and operational complexity.	
 Key Takeaway		Microservices are not a silver bullet. Use them when they solve real problems—complexity, scale, organizational agility. Design thoughtfully, start small, and evolve.		



Service-Oriented Architecture vs. Microservices

Both SOA and Microservices promote reuse and modularity, but they differ in design philosophy, governance, granularity, and operational independence.



Aspect	SOA	Microservices
Granularity	Coarse-grained services	Fine-grained, single responsibility
Governance	Centralized governance and standards	Decentralized governance, team-driven
Integration	ESB-centric, message mediation	Lightweight, decentralized integration
Data Management	Shared database and data model	Database per service, data ownership
Deployment	Deployed as larger, multi-service units	Independently deployable services
Scalability	Scale the entire application	Scale individual services independently
Change Agility	Changes impact multiple services and teams	Changes are isolated and faster to deliver



When to Use

SOA is a good fit when:

- ✓ You need enterprise-wide integration and reuse
- ✓ You have standardized processes and governance
- ✓ You have legacy systems to integrate
- ✓ You prefer a stable, centralized platform

Microservices is a good fit when:

- ✓ You need high agility and fast feature delivery
- ✓ Teams are autonomous and cross-functional
- ✓ You want independent scaling and deployment
- ✓ You embrace cloud-native and DevOps culture

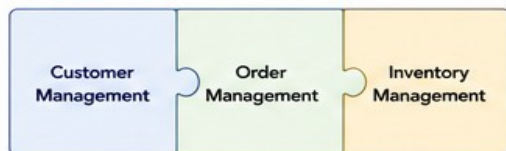


Key Takeaway: SOA focuses on integration and reuse through centralized control, while Microservices focus on agility, independent delivery, and decentralized ownership.

Core Principles of Microservices

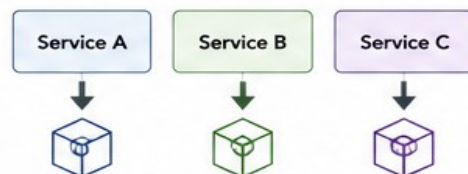
Microservices are built around a set of foundational principles that enable agility, resilience, scalability, and independent evolution of services.

1 Business Capability Alignment



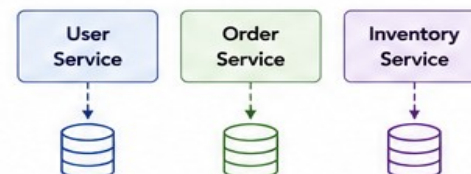
- Services are organized around specific business capabilities and domains.
- Reflects how the business operates.
- Enables faster delivery of business value.

2 Independent Deployability



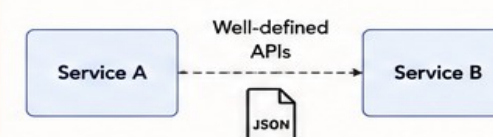
- Each service can be built, tested, and deployed independently.
- No need to redeploy the entire system.
- Enables continuous delivery and faster releases.

3 Decentralized Data Management



- Each service owns its data and database.
- Avoids data coupling and shared databases.
- Enables autonomy and scalability.

4 Loose Coupling



- Services communicate through well-defined contracts (APIs or events).
- Minimize dependencies between services.
- Changes in one service have minimal impact on others.

5 High Cohesion



- Each service focuses on a single responsibility and related functions.
- Keeps the service small, understandable, and maintainable.
- Encourages clean design and team focus.

6 Technology Heterogeneity



- Freedom to choose the right technology per service.
- Use the best tool for the job.
- Enables innovation and continuous improvement.

7 Resilience by Design



- Expect failures and design for them.
- Implement resilience patterns to contain failures.
- Ensure graceful degradation and recovery.

8 Observability Everywhere



- Instrument and monitor every service.
- Logs, metrics, and traces for visibility.
- Enables faster troubleshooting and better decisions.



Core Idea: These principles empower microservices to be **small, autonomous, resilient, and aligned to business value.**

Business-Capability Alignment

Design microservices around **what the business does**, not around *how the technology is built*.

WHAT IT MEANS

Align each microservice to a specific business capability that delivers value to customers. Capabilities are stable, understandable, and map to how the organization operates.

WHY IT MATTERS

- ✓ Focuses teams on business outcomes
- ✓ Improves agility and decision-making
- ✓ Helps prioritize investments
- ✓ Simplifies communication across the org
- ✓ Supports independent evolution and scaling

GOOD ALIGNMENT RESULTS IN

- ✓ Autonomous teams with clear purpose
- ✓ Services that evolve with the business
- ✓ Better customer experience
- ✓ Lower complexity and rework

KEY TAKEAWAY

Align microservices with business capabilities to deliver value faster, reduce complexity, and empower teams.

FROM BUSINESS CAPABILITIES TO MICROSERVICES

BUSINESS CAPABILITIES



CUSTOMER MANAGEMENT
Manage customer information, profiles, and preferences



ORDER MANAGEMENT
Manage orders from creation to fulfillment



PRODUCT CATALOG
Manage products, pricing, and availability



PAYMENT PROCESSING
Handle payments, refunds, and settlements



SHIPPING & DELIVERY
Manage shipments, tracking, and delivery



Design services around business capabilities.
Technology choices come later.

MICROSERVICES



Customer Service



Order Service



Catalog Service



Payment Service



Shipping Service

GUIDING PRINCIPLES

- ✓ **Start with the business**
Understand the domain and identify core business capabilities.
- ✓ **Stable and valuable**
Choose capabilities that are meaningful, valuable, and unlikely to change frequently.
- ✓ **Clear boundaries**
Each capability has a well-defined scope and ownership.
- ✓ **Autonomous ownership**
Cross-functional team owns the service end-to-end.
- ✓ **Independent evolution**
Services evolve, scale, and deploy independently.

EXAMPLE: E-COMMERCE PLATFORM

CAPABILITIES

- Customer Management
- Order Management
- Product Catalog
- Payment Processing
- Shipping & Delivery
- Returns Management
- Notification Management

MICROSERVICES

- customer-service
- order-service
- catalog-service
- payment-service
- shipping-service
- returns-service
- notification-service

SUCCESS STARTS WITH ALIGNMENT



Understand the business



Identify core capabilities



Define clear boundaries



Build autonomous services



Deliver value continuously

INDEPENDENTLY DEPLOYABLE SERVICES

Each microservice deploys independently, unlocking faster delivery and reduced release risk.



TRADITIONAL MONOLITH VS. MICROSERVICES

- **Monolith:** one deployable unit; any change requires redeploying the entire application
- **Microservices:** each service (User, Order, Inventory, Billing) is deployed independently
- Changes in one service do not impact others



HOW IT WORKS (5 STEPS)

- 1. Develop:** teams work on their respective services
- 2. Test:** each service is tested in isolation and in pipeline
- 3. Package:** service is packaged as an artifact or container image
- 4. Deploy:** service is deployed to its runtime environment
- 5. Operate:** monitor, scale, and update the service independently



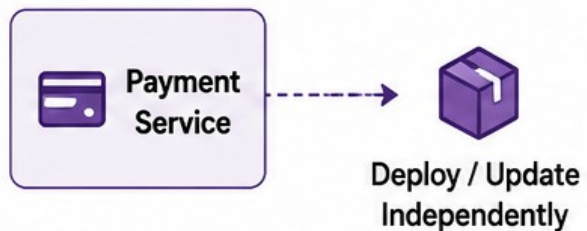
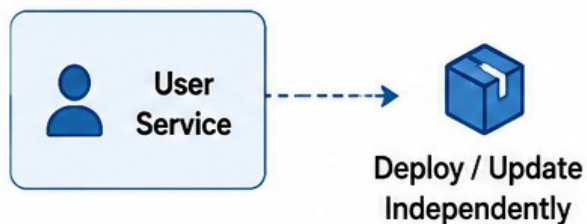
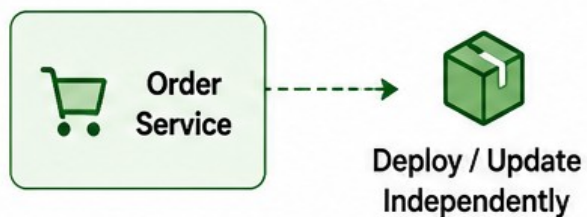
KEY BENEFITS

- **Faster Delivery** - deploy small changes quickly without waiting for other teams or services
- **Reduced Risk** - issues in one service do not require redeploying the entire application
- **Team Autonomy** - teams own their services end-to-end and choose the best tools
- **Scalability** - scale only the services that need more capacity
- **Continuous Improvement** - services evolve independently with minimal coordination

KEY TAKEAWAY Independently deployable services enable agility, resilience, and faster time to value.

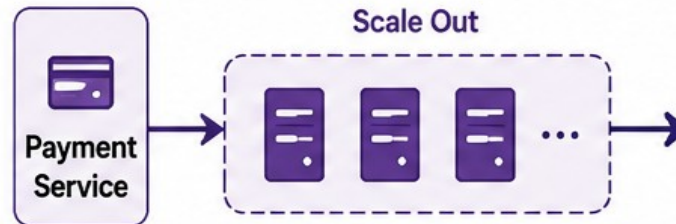
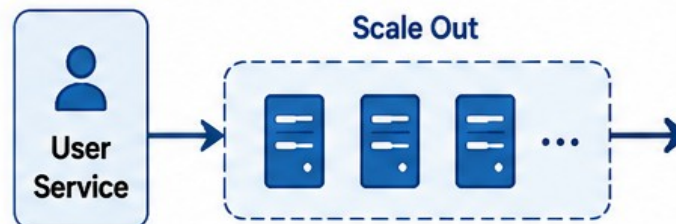
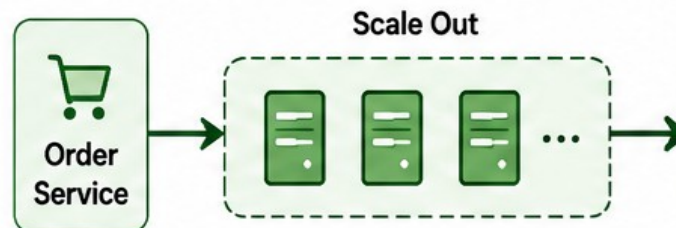
Independent Deployment

Each service can be developed, deployed, and updated independently without affecting others.



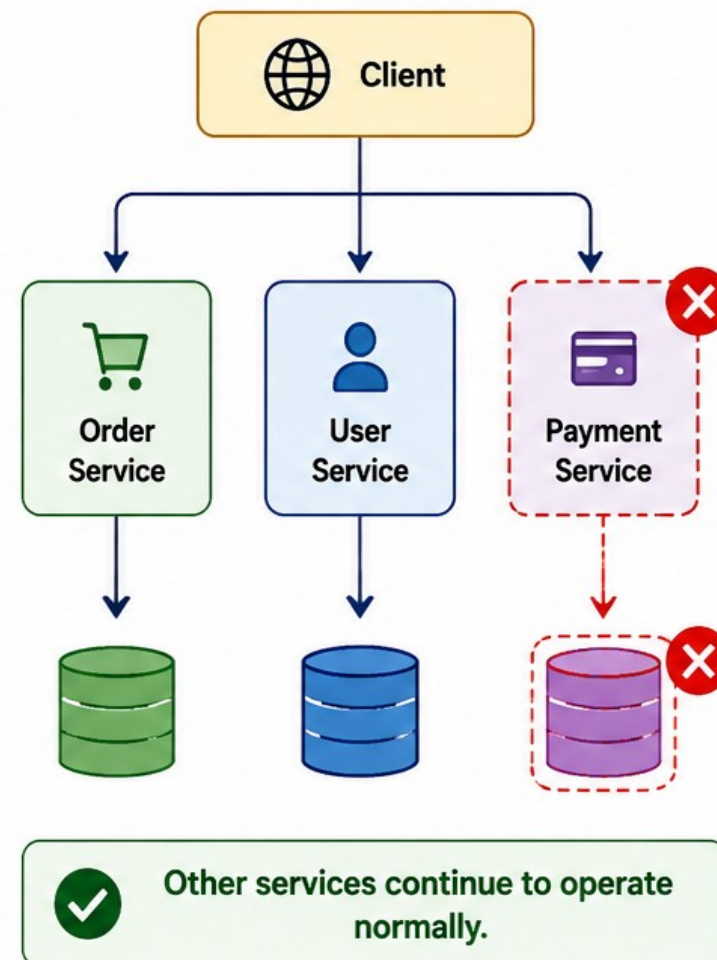
Scalability

Services can scale independently based on demand.



Fault Isolation

Failure in one service does not bring down the entire system.



DECENTRALIZED OWNERSHIP AND GOVERNANCE

Ownership sits with cross-functional teams; lightweight governance keeps them aligned.



DECENTRALIZED OWNERSHIP

- Each service owned end-to-end by a cross-functional team with autonomy and accountability
- Team designs, builds, tests, deploys, operates, and evolves its service
- End-to-end accountability - "You build it, you run it"
- Faster decisions - local context leads to better choices
- Empowered teams and clear boundaries - teams own the service and its data



DECENTRALIZED GOVERNANCE

- Establish a minimal set of standards/guardrails all teams follow, without a central bottleneck
- Define the guardrails, not the implementation
- Enable consistency through shared platform services
- Govern by principles, automate compliance
- Review and evolve standards continuously

KEY TAKEAWAY Decentralized ownership drives speed and innovation. Lightweight governance provides alignment, safety, and consistency at scale.

Loose Coupling and High Cohesion

Microservices are effective when services are **loosely coupled** to each other and **highly cohesive** within themselves.

Loose Coupling

Services minimize dependencies on each other.

Example Architecture

Principles

- ✓ Communicate through well-defined contracts (APIs or events).
- ✓ Avoid direct access to another service's data or implementation.
- ✓ Changes in one service have minimal impact on others.
- ✓ Use asynchronous messaging for non-critical workflows.
- ✓ Depend on abstractions, not concrete implementations.

Benefits

- Independent Deployment
- Resilience Improved
- Technology Flexibility
- Faster Change
- Team Autonomy

High Cohesion

Sach service focuses on a single, well-defined responsibility.

Example: Order Service (Highly Cohesive)

Principles

- ✓ Group related data and behaviors that belong together.
- ✓ Keep the service small and focused on one business capability.
- ✓ Avoid adding unrelated features to a service.
- ✓ Easier to understand, develop, test, and maintain.

Benefits

- Focus and Clarity
- Easier Maintenance
- Better Testability
- Scalability by Business Need
- Lower Risk of Change

Best Practices

- Design and version stable APIs.
- Prefer events for integration.
- Never share database or schemas.
- Use contracts and schema validation.
- Monitor dependencies and coupling.
- Review boundaries regularly.



Core Idea: Loosely coupled services can change independently. **Highly cohesive services are simpler, safer, and more effective.**

Technology and Data Decentralization

Decentralize technology choices and data ownership to build **independent**, **resilient**, and **evolvable** microservices.

WHAT IT MEANS

Each microservice is free to choose the best technology stack and owns its data end-to-end. This eliminates tight coupling and enables teams to move at their own pace.

WHY IT MATTERS

- Teams can use the right tool for the job
- Independent evolution and innovation
- Fault isolation and reduced blast radius
- Scales organization and technology diversity
- Avoids shared-database bottlenecks

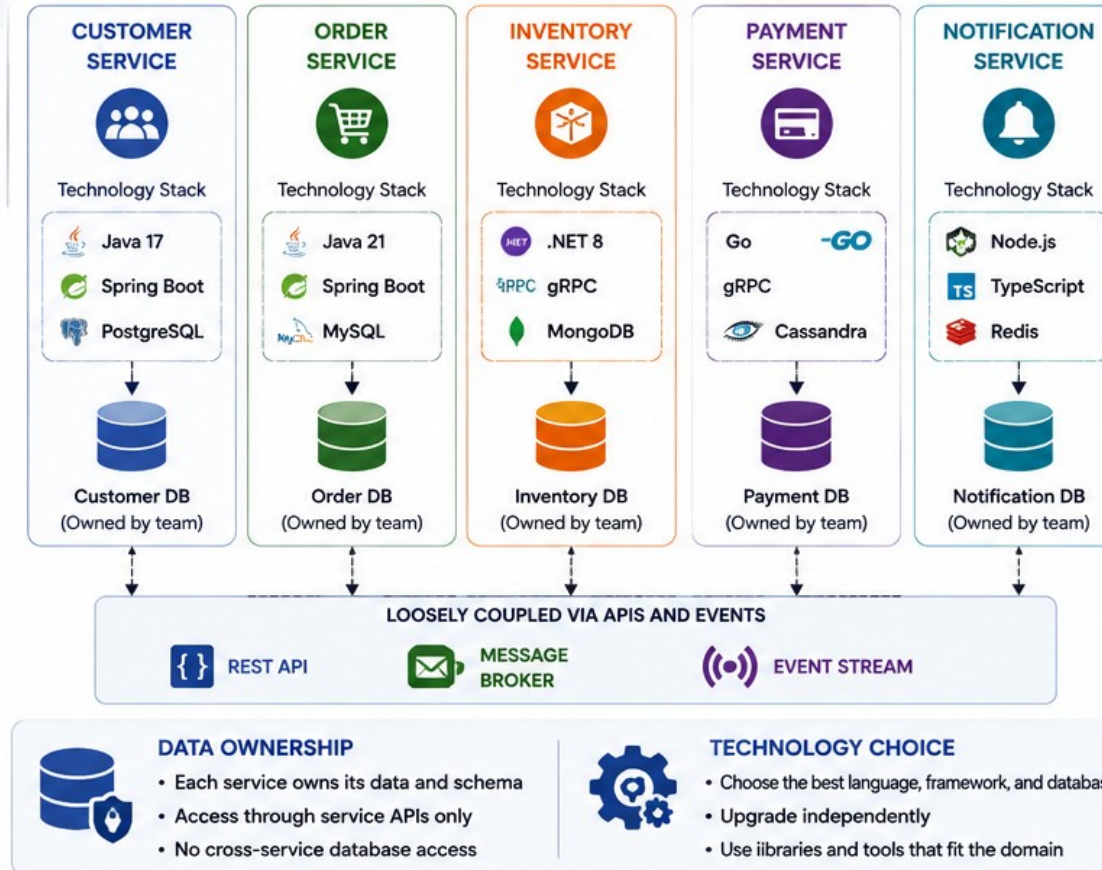
AVOID THESE ANTI-PATTERNS

- Shared databases across services
- One-size-fits-all technology stack
- Centralized data access libraries
- Leaky abstractions and tight coupling

KEY TAKEAWAY

Decentralizing technology and data empowers teams, reduces coupling, and enables scalable, resilient microservices that evolve with the business.

DECENTRALIZED TECHNOLOGY AND DATA MODEL



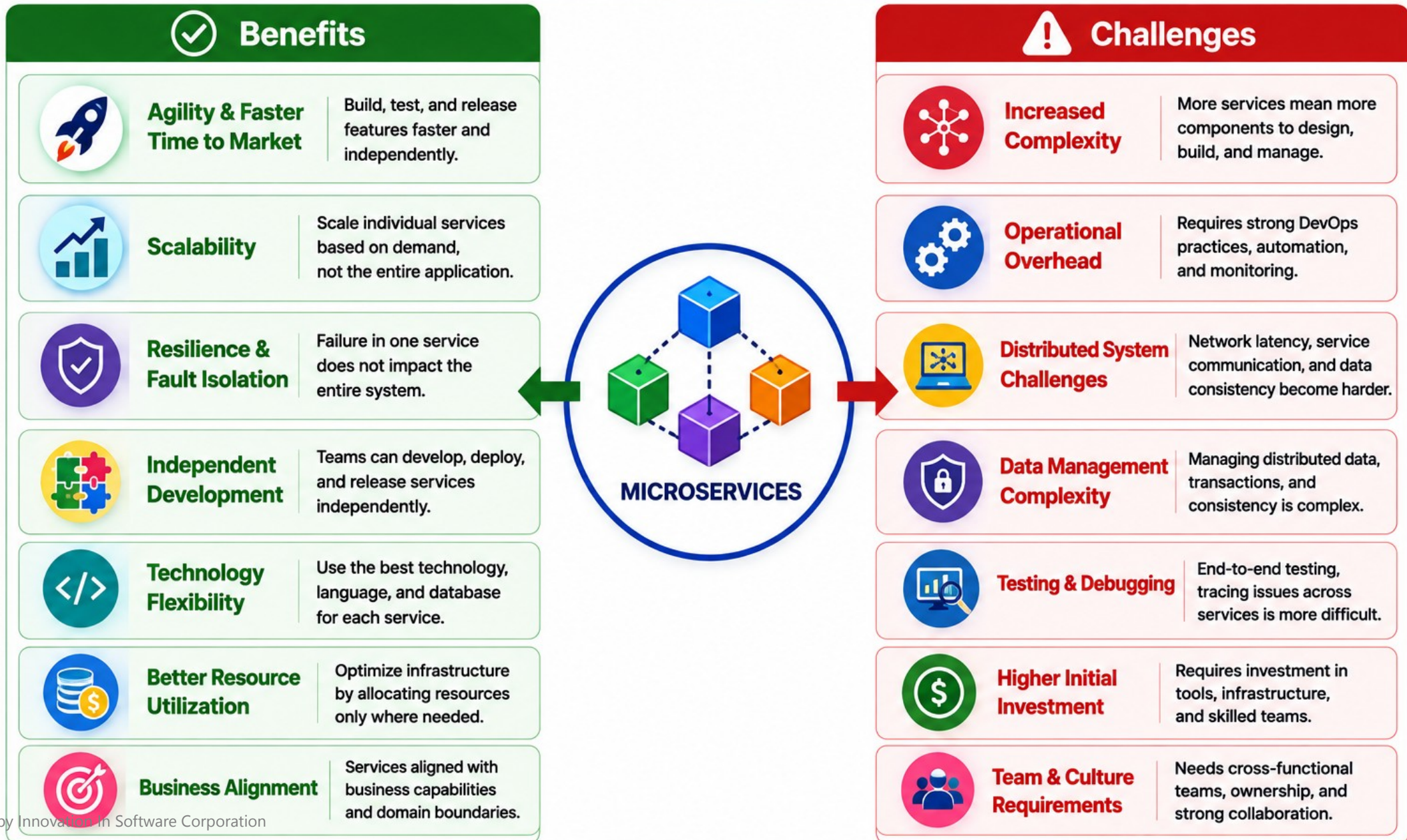
KEY PRACTICES

- Database per Service**
Each service has its own database and schema.
- Technology Diversity**
Use the best technology for each business capability.
- Access via APIs**
Communicate through well-defined APIs or events.
- Clear Boundaries**
Enforce service boundaries for data and technology.
- Evolve Independently**
Deploy, scale, and upgrade without affecting others.
- Observe and Govern**
Monitor usage, performance, and data contracts.

OUTCOMES YOU ACHIEVE

- Better fit for business needs**
- Faster delivery and innovation**
- Improved resilience and fault isolation**
- Teams empowered and accountable**
- Scalable organization and technology**

Benefits and Challenges



Operational Complexity in Distributed Systems

Distributed systems introduce **new operational challenges** that do not exist in monoliths. They require more **moving parts**, more **coordination**, and more **discipline**.

WHY COMPLEXITY INCREASES



Many Independent Services

Each service has its own codebase, runtime, team, and lifecycle.



Distributed Runtime

Services run across multiple hosts, zones, or regions.



Network Communication

Every call can fail, timeout, or be slow. Failures are partial, not total.



Data Decentralization

Multiple databases and data stores must be managed independently.



Multiple Teams

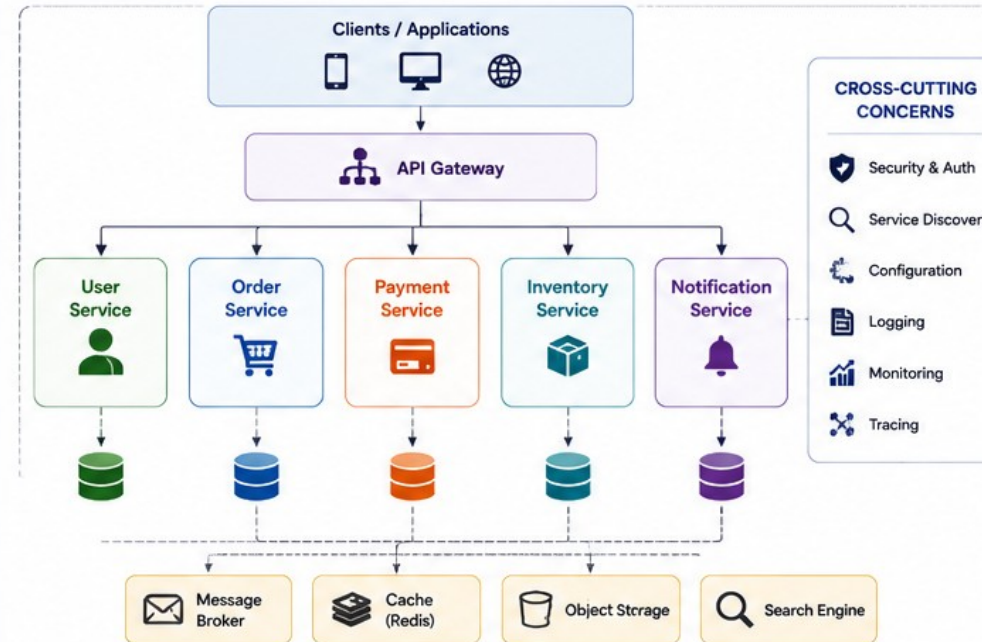
Coordination across teams, priorities, and processes adds overhead.



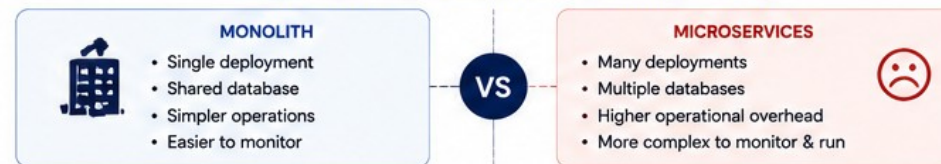
Operational Tooling

More infrastructure, monitoring, pipelines, and automation to manage.

A TYPICAL MICROSERVICES ENVIRONMENT



MONOLITH vs. MICROSERVICES (OPERATIONAL VIEW)



OPERATIONAL CHALLENGES



Partial Failures

A failure in one service affects only some requests, making detection harder.



Latency and Timeouts

Network latency, retries, and timeouts affect user experience.



Complex Dependencies

Many service-to-service and external dependencies to manage.



Observability Complexity

Collecting and correlating logs, metrics, and traces across services is harder.



Data Consistency

Maintaining data consistency across multiple services is challenging.



Deployment Complexity

Coordinating deployments across many services and environments.



Cost Management

More infrastructure, tooling, and operational overhead increase costs.

REALITY CHECK



5-10x

More services to deploy and monitor in production



2-5x

More infrastructure resources for the same business capability



Higher MTTR

Troubleshooting takes longer without good observability



KEY TAKEAWAY

Microservices shift complexity from development to operations. Success depends on automation, observability, and disciplined engineering practices.

MANAGE COMPLEXITY WITH



Automation (CI/CD, IaC)



Observability (Logs, Metrics, Traces)



Resilience Patterns (Timeouts, Retries, Circuit Breakers)



Standardization (Templates, Libraries, Best Practices)



Clear Ownership and Governance



Continuous Testing and Verification

Costs and Tradeoffs of Microservices

Microservices deliver significant benefits, but they also introduce **costs** and **tradeoffs** that must be understood and managed.

THE COSTS

-  **1. Operational Complexity**
Operating many services, environments, and dependencies increases complexity.
-  **2. Distributed System Challenges**
Network latency, partial failures, timeouts, and eventual consistency add complexity.
-  **3. Infrastructure and Tooling Overhead**
More infrastructure, observability, CI/CD, and security tooling are required.
-  **4. Testing Complexity**
More services mean more tests, environments, and data setup for reliable testing.
-  **5. Organizational Overhead**
More coordination, communication, and governance across teams is required.
-  **6. Higher Cost at Small Scale**
At low scale, the overhead may outweigh the benefits of microservices.



THE TRADEOFFS

-  **1. Speed vs. Consistency**
Faster independent releases may come with eventual consistency and complexity.
-  **2. Autonomy vs. Standardization**
Team freedom can lead to duplicated efforts and inconsistent approaches.
-  **3. Smaller Services vs. Overhead**
Finer-grained services improve agility but increase communication and ops overhead.
-  **4. Flexibility vs. Governance**
Technology freedom improves fit but requires strong governance to control sprawl.
-  **5. Decentralization vs. Coordination**
Local decision-making empowers teams, but cross-service coordination is essential.
-  **6. Short-Term Productivity vs. Long-Term Value**
Initial learning curve and setup effort can slow teams before realizing long-term benefits.



HOW TO MANAGE THESE COSTS



Start with the Right Scope

Decompose by business capability, not by technology layers.



Automate Everything

Use CI/CD, infrastructure as code, automated testing, and self-service platforms.



Invest in Observability

Centralized logs, metrics, traces, and alerting are essential.



Empower Teams with Guardrails

Provide standards, templates, and platform services.



Security by Design

Implement consistent security patterns and automated checks.



Continuously Evaluate

Monitor costs and benefits, and evolve architecture over time.



KEY TAKEAWAY

Microservices are not free. They trade simplicity for flexibility, speed, and resilience. Understand the costs, plan for them, and use automation and good architecture to stay in control.

WHEN MICROSERVICES MAKE SENSE



Multiple teams working on different domains



Need for independent deployment and scaling



Application is growing in complexity and scale



Need technology diversity and flexibility



High availability and resilience requirements



Long-term business value outweighs initial costs



The goal is not to build more microservices, but to build the right ones for the right reasons.

ORGANIZATIONAL IMPLICATIONS OF MICROSERVICES

Microservices demand organizational change: decentralized ownership, new skills, and evolved culture.



SHIFT TO DECENTRALIZED OWNERSHIP / TEAM STRUCTURE / CULTURE

1. Shift to Decentralized Ownership - services owned end-to-end by small, cross-functional teams
 - Teams make decisions, deploy independently, and own the operational lifecycle
2. Team Structure Evolution - move from siloed departments to autonomous, product-aligned teams
3. **Culture and Mindset Change - embrace DevOps culture:** "You build it, you run it"



GOVERNANCE AND ALIGNMENT / NEW SKILLS / INVESTMENT

4. Governance and Alignment - establish lightweight governance with clear guardrails and standards
 - Define API standards, security policies, platform guidelines; balance autonomy with consistency
5. New Skills and Investment - teams need strong DevOps, automation, cloud-native, and observability skills
 - Invest in training, tooling, and communities of practice



FROM CENTRALIZED TO DECENTRALIZED

- **Traditional/Monolithic:** Centralized IT with Dev, Ops, QA, Business silos
- **Cons:** siloed teams and handoffs, shared priorities and queues, slower delivery and innovation
- **Microservices Organization:** Service A-D each owned by a cross-functional team
- **Pros:** autonomous, empowered teams; aligned to business capabilities; faster value and customer focus



KEY ORGANIZATIONAL OUTCOMES

- Faster delivery of business value
- Stronger alignment with business capabilities
- Higher team autonomy and morale
- Improved resilience and adaptability
- Better efficiency over time
- Sustainable innovation and competitive advantage

KEY TAKEAWAY Microservices require an organizational evolution. Success depends on empowered teams, clear guardrails, the right culture, and investing in people and platforms.

MICROSERVICES ANTI-PATTERNS

Avoid these common traps that can undermine the benefits of microservices.



COMMON ANTI-PATTERNS

1. **Too Many Services:** creating a service for every feature or entity leads to unnecessary complexity
2. **Shared Database:** sharing a database breaks service independence and creates tight coupling
3. **Synchronous Everything:** relying only on synchronous calls increases latency and limits scalability
4. **Chatty Services:** one user action fans out into many small calls, multiplying latency and failure points
5. **Anemic Services:** a service that only forwards CRUD to a table carries cost without owning any behaviour
6. **Centralized Data or Business Logic:** a shared database or rules engine recreates the monolith behind the services
7. **Big Bang Migration:** rewriting everything at once maximises risk and delays any delivered value
8. **Distributed Monolith:** services that must deploy together give you network cost with none of the autonomy
9. **Over-Engineering Resilience:** retries and breakers everywhere add complexity and can amplify load
10. **Ignoring Observability:** without logs, metrics, and traces a distributed failure is effectively undebuggable

KEY TAKEAWAY Microservices amplify both good design and bad design. Avoid these anti-patterns to realize the true benefits of agility, resilience, and scale.

MICROSERVICES ANTI-PATTERNS

Avoid these common traps that can undermine the benefits of microservices.

WHY THEY'RE DANGEROUS

- Increase complexity and operational costs
- Reduce performance and scalability
- Create tight coupling and hidden dependencies
- Increase failure risk and blast radius
- Slow down teams and innovation

BEST PRACTICES TO AVOID THEM

- Start with business capabilities, not technology
- Keep services small, cohesive, and autonomous
- Own your data and avoid sharing
- Use the right communication pattern
- Invest in observability from the start; evolve incrementally and learn continuously

KEY TAKEAWAY Microservices amplify both good design and bad design. Avoid these anti-patterns to realize the true benefits of agility, resilience, and scale.

MICROSERVICES AND CONWAY'S LAW

Organizations design systems that mirror their communication structures.

CONWAY'S LAW

- "Organizations which design systems are constrained to produce designs which are copies of the communication structures of these organizations." -- Melvin Conway
- **Key Insight:** if your organization is siloed, your microservices will be too
- Align team boundaries, communication, and ownership with service boundaries

HOW ORGANIZATION STRUCTURE SHAPES YOUR ARCHITECTURE

- Siloed Organization (Sales, Marketing, Billing, Support silos): communication limited to silos -> Likely Outcome: Poorly Designed Microservices, high coupling, duplicated functionality
- **Partially Aligned Organization:** some collaboration but unclear boundaries -> Likely Outcome: Inconsistent Services
- **Aligned Organization:** teams aligned to business capabilities -> Likely Outcome: Well-Designed Microservices, loosely coupled, independently deployable

KEY TAKEAWAY Good microservices architecture starts with an aligned organization. Align teams to business capabilities, establish clear communication, and design services that enable autonomy, speed, and resilience.

MICROSERVICES AND CONWAY'S LAW

Organizations design systems that mirror their communication structures.

WHY IT MATTERS

- **Alignment:** aligned teams create cohesive services that map to real business capabilities
- **Autonomy:** teams can build, deploy, and evolve services independently
- **Speed:** clear boundaries reduce coordination overhead and speed up delivery
- **Quality:** ownership and accountability improve code quality; Scalability by business need

WATCH OUT FOR

- Creating services that mirror org charts, not domains
- Siloed teams building tightly coupled services
- Interfaces designed for internal teams, not capabilities
- Frequent coordination across "independent" services

KEY TAKEAWAY Good microservices architecture starts with an aligned organization. Align teams to business capabilities, establish clear communication, and design services that enable autonomy, speed, and resilience.

Team Topologies for Microservices

Organizing teams to maximize flow, alignment, and platform excellence

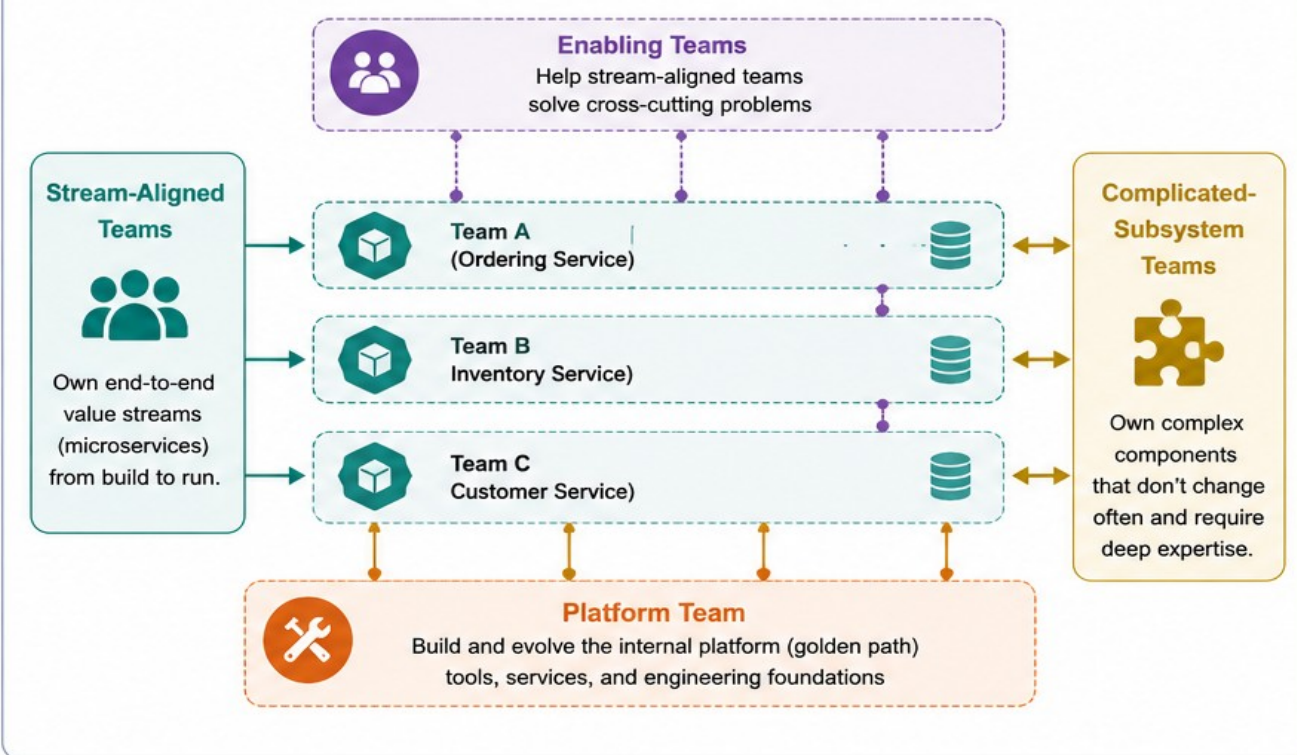
Effective team structure is as important as technical architecture. Team Topologies aligns teams with business value and system architecture to enable fast flow, clear ownership, and sustainable delivery of microservices.



Goals

- Deliver business value quickly and reliably
- Enable strong collaboration and autonomy
- Provide a robust platform for teams to build on
- Reduce cognitive load and coordination overhead

TEAM TOPOLOGIES MODEL



TEAM TYPES



Stream-Aligned Teams

Cross-functional teams aligned to business capabilities and value streams. Own their microservices end-to-end including operations and support.



Enabling Teams

Provide expertise, components, and guidance to stream-aligned teams. Focus on cross-cutting concerns such as authentication, observability, testing frameworks, or developer productivity.



Platform Team

Builds and maintains the internal platform, tools, CI/CD pipelines, infrastructure templates, and standards that streamline delivery and reduce cognitive load.



Complicated-Subsystem Teams

Own components that are too complex or risky for stream teams and change infrequently (e.g., legacy systems, high-risk domains, advanced analytics engines).



Align with value

Organize around business capabilities and value streams.



Minimize team interactions

Reduce dependencies and handoffs.



Provide a paved road

Platform teams make the easy path the best path.



Measure flow

Optimize for flow speed, quality, and feedback loops.



Evolve continuously

Regularly review team structure and adjust as your architecture grows.

DEVOPS OWNERSHIP AND "YOU BUILD IT, YOU RUN IT"

Empowering teams with end-to-end responsibility for value and reliability.



THE PRINCIPLE IN ONE LINE

- "When teams own their services end-to-end, they design better, release with confidence, and operate with excellence."



END-TO-END OWNERSHIP LIFECYCLE

- 1. Plan:** understand business needs, define boundaries and APIs, plan for reliability, security, and scale
- 2. Build:** develop with quality and best practices, write automated tests, secure coding
- 3. Release:** automate build and tests in CI/CD, package and version the service, deploy to environments consistently
- 4. Operate:** monitor health, metrics, and logs; ensure performance, availability, and capacity
- 5. Improve:** analyze feedback and telemetry, run blameless postmortems, continuously deliver better value

KEY TAKEAWAY You Own It. You Operate It. You Improve It. End-to-end ownership builds better software, more resilient systems, and stronger teams.

DEVOPS OWNERSHIP AND "YOU BUILD IT, YOU RUN IT"

Empowering teams with end-to-end responsibility for value and reliability.



WHY IT MATTERS

- **Higher Quality:** teams build reliable services because they experience the impact in production
- **Faster Feedback:** close to operations, teams get rapid feedback and iterate quickly
- **Greater Accountability:** clear ownership eliminates handoffs and unclear responsibility
- **Lower Cost of Change:** well-operated services have fewer incidents and lower rework



KEY PRACTICES

- **Automate:** automate build, test, deploy, and rollback pipelines
- **Observe:** use logs, metrics, and traces to gain real-time insight
- **Respond:** detect issues early and respond quickly and effectively
- **Learn:** run postmortems, share learnings, and improve
- **Secure:** build security in from the start and continuously scan

KEY TAKEAWAY You Own It. You Operate It. You Improve It. End-to-end ownership builds better software, more resilient systems, and stronger teams.

Microservices in Modern Cloud-Native Platforms

Build small. Deploy anywhere. Scale elastically. Operate resiliently.

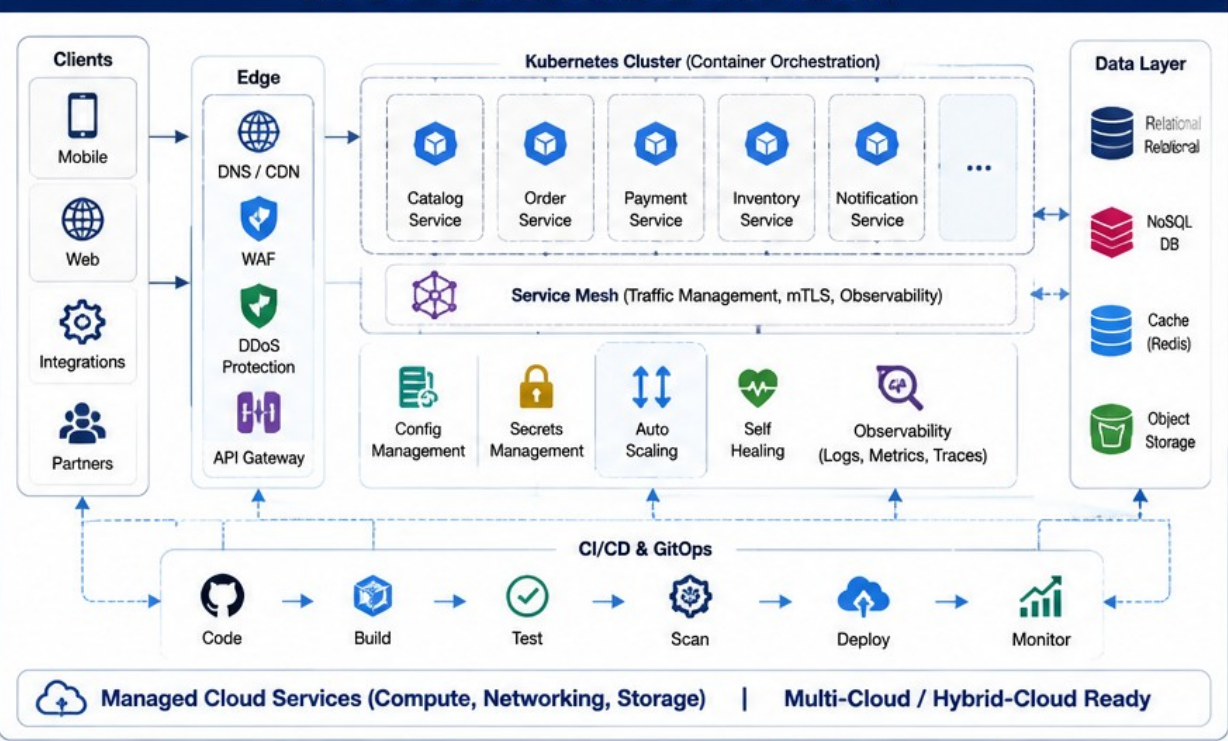
Cloud-native platforms provide the infrastructure, services, and automation that microservices need to deliver business value faster—while improving scalability, resilience, and operational efficiency.



Cloud-Native Value

- ✓ Elastic scale on demand
- ✓ Resilient by design
- ✓ Faster time to market
- ✓ Continuous delivery at scale
- ✓ Pay for use and optimize costs

CLOUD-NATIVE MICROSERVICES ARCHITECTURE



WHY MICROSERVICES THRIVE IN CLOUD-NATIVE PLATFORMS

- Elastic Scalability** – Scale each service independently based on demand.
- Resilience** – Built-in health checks, self-healing, and automated recovery.
- Speed** – Automated provisioning and pipelines enable rapid delivery.
- Cost Efficiency** – Right-size resources and pay only for what you use.
- Security** – Identity, secrets, network policies, and compliance by design.
- Observability** – Unified logs, metrics, and traces across distributed services.

CLOUD-NATIVE DESIGN PRINCIPLES

- Loosely Coupled**
Independent deployment and evolution
- Stateless Services**
Any instance can handle any request
- Resilient by Design**
Expect failure, recover automatically
- Automate Everything**
Infrastructure as code and policy as code
- Secure by Default**
Zero trust, least privilege, encryption
- Observe & Improve**
Measure, learn, and iterate

KEY CLOUD-NATIVE ENABLERS

- Container Orchestration**
Deploy, scale, and manage containers automatically.
- Service Mesh**
Secure, observe, and control service communication.
- CI/CD & GitOps**
Automate delivery and drift-free operations.
- Observability Suite**
Collect logs, metrics, and traces in one place.
- Managed Services**
Leverage databases, queues, storage, and AI/ML services.
- Multi-Cloud Flexibility**
Avoid lock-in and run where it makes sense.

BUSINESS OUTCOMES

- Faster Innovation**
Deliver features and experiments faster.
- Higher Availability**
Resilient services deliver better experiences.
- Better Customer Experience**
Reliable, responsive, and personalized services.
- Operational Excellence**
Automate, observe, and optimize continuously.
- Business Agility**
Adapt quickly to change and grow.

Distributed System Fundamentals

1. DISTRIBUTION



Components run on multiple machines



Communicate over a network



Cooperate to achieve a common goal

2. TRANSPARENCY



Hide the complexity of distribution from users

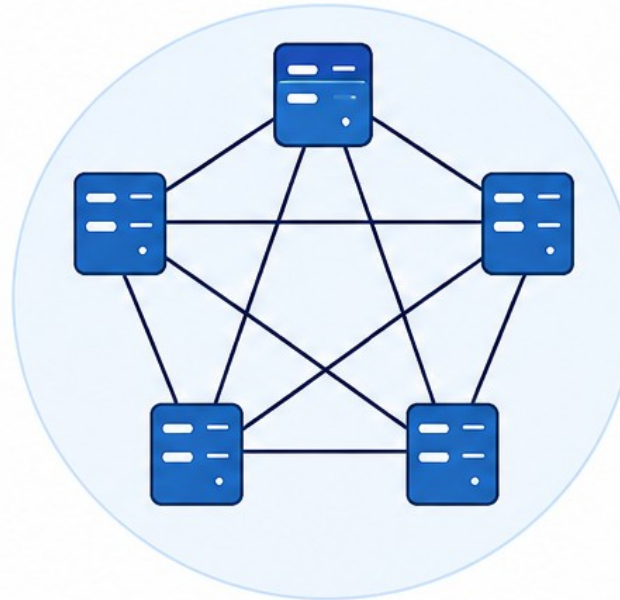


Access resources as if local



Examples: access, location, migration, replication, concurrency, failure transparency

A distributed system is a collection of independent computers that appear to users as a single coherent system.



5. CORE CONCEPTS



Communication
(IPC, Messaging)



Data
Replication



Consistency
Models



Fault
Tolerance



System
Model

3. KEY PROPERTIES



Scalability – Handle growth



Availability – Remain operational



Reliability – Correct service despite failures



Performance – Efficient resource use



Consistency – Correct and up-to-date data

4. COMMON CHALLENGES



Network latency and delays



Partial failures



Network partitions



Concurrency and coordination



Security across nodes

INTRODUCTION TO THE TWELVE-FACTOR APP

A methodology for building modern, scalable, and maintainable cloud-native applications.



WHAT IS THE TWELVE-FACTOR APP?

- A set of principles for building applications that run reliably in modern cloud environments
- Scale up without significant changes
- Are easy to deploy, maintain, and evolve
- Are independent of underlying infrastructure



IN THEIR OWN WORDS

- "The Twelve-Factor App is not a rigid prescription, but a collection of principles that work together to help you build better software." -- 12factor.net



CORE IDEAS

- **Codebase:** one codebase, many deploys
- **Config Externalized:** store config in the environment
- **Scalability:** scale out via concurrency
- **Observability:** logs, metrics, health checks
- **Dev & Ops Alignment:** best practices for teams
- **Infrastructure Agnostic:** run on any modern platform

KEY TAKEAWAY The Twelve-Factor App is a foundation for modern software engineering and DevOps excellence. The goal: focus on code and business value, not on infrastructure.

INTRODUCTION TO THE TWELVE-FACTOR APP

A methodology for building modern, scalable, and maintainable cloud-native applications.

 THE TWELVE FACTORS (1/2)

ITEM	DETAIL
1. One Codebase	One codebase tracked in revision control, multiple deployments.
2. Dependencies	Explicitly declare and isolate dependencies.
3. Config	Store config in the environment, not in code.
4. Backing Services	Treat backing services as attached resources.
5. Build, Release, Run	Strictly separate build and release from run.
6. Processes	Execute the app as one or more stateless processes.

KEY TAKEAWAY The Twelve-Factor App is a foundation for modern software engineering and DevOps excellence. The goal: focus on code and business value, not on infrastructure.

INTRODUCTION TO THE TWELVE-FACTOR APP

A methodology for building modern, scalable, and maintainable cloud-native applications.

 THE TWELVE FACTORS (2/2)

ITEM	DETAIL
7. Port Binding	Export services via port binding.
8. Concurrency	Scale out via the process model.
9. Disposability	Maximize robustness with fast startup and graceful shutdown.
10. Dev/Prod Parity	Keep development, staging, and production as similar as possible.
11. Logs	Treat logs as event streams.
12. Admin Processes	Run admin/management tasks as one-off processes.

 KEY BENEFITS

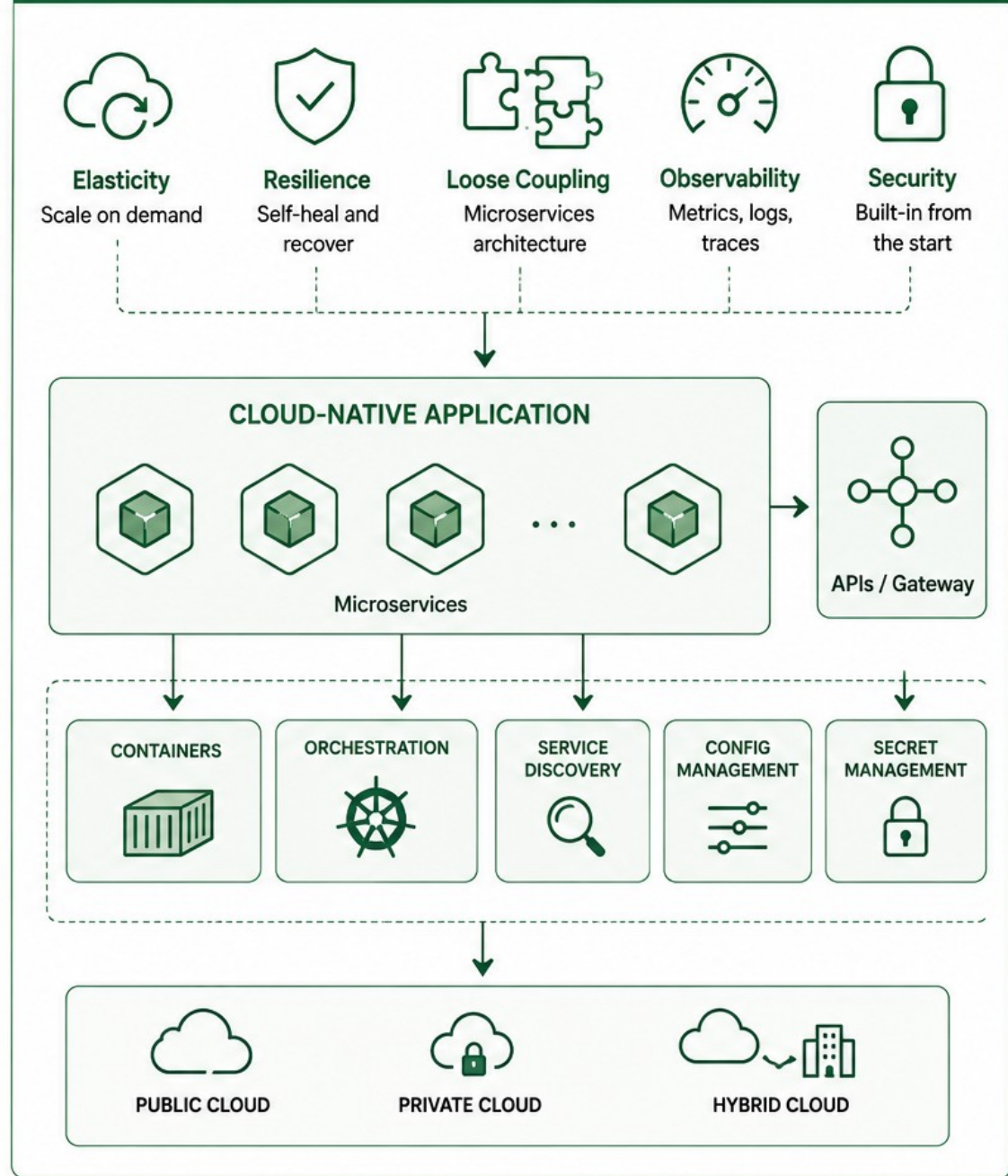
- **Built for the Cloud:** optimized for dynamic, scalable cloud environments
- **Resilient & Scalable:** designed for failure, recovery, and horizontal scale
- **Faster Delivery:** easier to build, test, deploy, and release frequently
- **Easier Operations and Portability:** run anywhere -- cloud, on-prem, or managed platforms

KEY TAKEAWAY The Twelve-Factor App is a foundation for modern software engineering and DevOps excellence. The goal: focus on code and business value, not on infrastructure.

TWELVE-FACTOR APP



CLOUD-NATIVE OVERVIEW



One Codebase, Multiple Deployments

Keep one codebase in version control and deploy it everywhere.

The first factor of the Twelve-Factor App methodology states that a single codebase should be used across all environments. This eliminates configuration drift, reduces complexity, and ensures consistency from development to production.

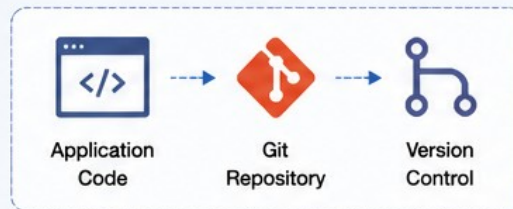


Key Idea

A single codebase goes through many stages and environments with different configurations, not different code.

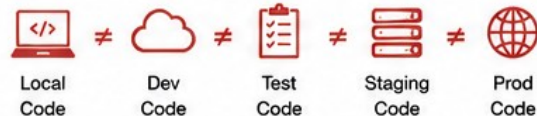
THE PRINCIPLE

One Codebase



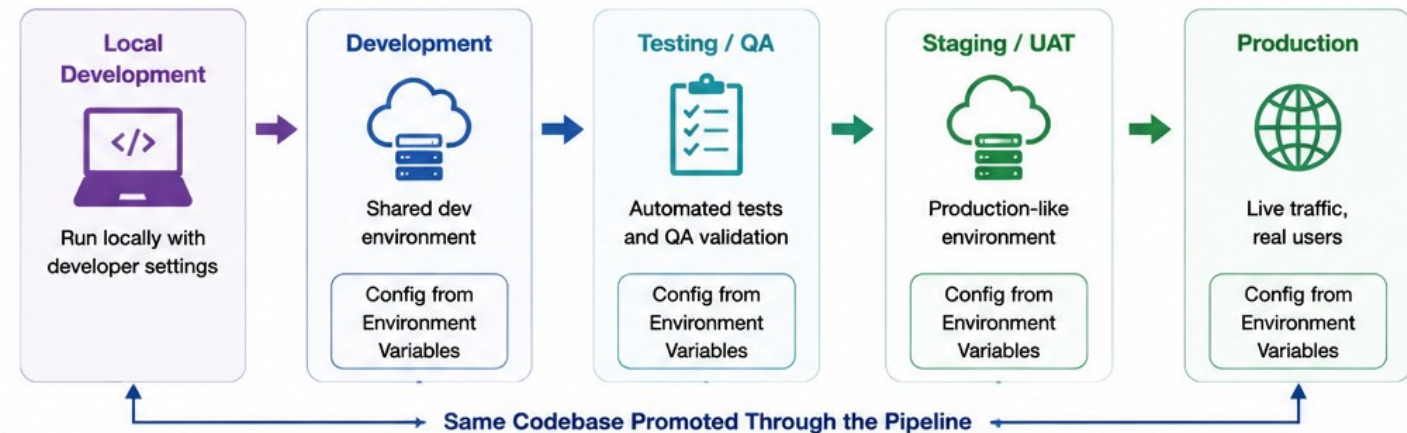
- ✓ Same code promoted through all environments
- ✓ Environment-specific config in the environment, not in the code
- ✓ Built once, deployed many times
- ✓ Easier to test, release, and roll back
- ✓ Reduced risk and operational overhead

AVOID: MULTIPLE CODEBASES



- ✗ Code divergence and configuration drift
- ✗ Harder to maintain and debug
- ✗ Higher risk of deployment failures

ONE CODEBASE PROMOTED ACROSS ENVIRONMENTS



HOW IT WORKS



Configuration Comes From the Environment



BENEFITS

- Consistency**
Same behavior across environments
- Speed**
Faster delivery and easier rollbacks
- Reliability**
Fewer surprises in production
- Team Productivity**
Less effort maintaining multiple versions
- Focus**
Focus on building value, not on managing code variations



One codebase. Many environments. Infinite possibilities.



Build once. Deploy anywhere. Scale everywhere.

EXPLICIT DEPENDENCY MANAGEMENT

Declare and isolate dependencies to ensure reliability and portability.

KEY PRINCIPLE & WHY IT MATTERS

- Declare all dependencies explicitly and completely in a dependency manifest
- Never rely on libraries or tools implicitly provided by the execution environment
- **Consistency:** same versions everywhere reduce unexpected behavior
- **Portability:** run anywhere without depending on system-installed libraries
- Reproducibility, maintainability, and security (easier vulnerability scanning)

WHAT IT LOOKS LIKE

- **NOT EXPLICIT (BAD):** System/Server provides Logging Lib, JSON Lib, DB Driver implicitly to Application -- Inconsistent & Unreliable
- **EXPLICIT (GOOD):** Application declares all dependencies (Logging Lib, JSON Lib, DB Driver); Runtime/Environment provides only execution -- Consistent & Portable

KEY TAKEAWAY Declare it. Isolate it. Control it. Explicit dependency management builds reliable, portable, and maintainable applications.

EXPLICIT DEPENDENCY MANAGEMENT

Declare and isolate dependencies to ensure reliability and portability.



COMMON MANIFESTS

- **Maven:** pom.xml -- XML based, widely used in Java ecosystem
- **Gradle:** build.gradle -- Groovy / Kotlin DSL, flexible and powerful
- **Node.js:** package.json -- declares npm packages and versions
- **Python:** requirements.txt -- list all Python packages with versions
- **Go:** go.mod -- Go modules, declare and track dependencies



BEST PRACTICES & RISKS OF NOT MANAGING EXPLICITLY

- **Do:** declare every dependency your app needs; pin versions, avoid "latest"
- **Do:** use dependency management for transitive deps; keep dependencies minimal
- **Do:** regularly update and scan for vulnerabilities; lock deps for reproducible builds
- **Do:** test in clean environments (no local assumptions)
- **Risks if not managed:** works on one machine fails on another, hidden system libraries cause runtime issues, harder upgrades and migrations

KEY TAKEAWAY Declare it. Isolate it. Control it. Explicit dependency management builds reliable, portable, and maintainable applications.

Configuration in the Environment

Store config in the environment, not in the code.

The third factor of the Twelve-Factor App states that all configuration must be stored in the environment. This keeps code open source-ready, promotes portability, and ensures the same codebase can run in any environment without changes.

“ Store config in the environment to separate code from config. This enables portability, security, and flexible deployments.



THE PRINCIPLE



Keep all configuration in the environment (env vars, config stores, etc.), not in code.

Configuration includes:

- Resource handles (DB, Redis, Queues)
- Credentials (users, passwords, API keys)
- Service endpoints and URLs
- Feature flags and tuning parameters

CONFIGURATION: BAD vs GOOD



HARD-CODED (BAD)

Configuration in code

```
# application.properties
spring.datasource.url=jdbc:mysql://localhost:3306/app
spring.datasource.username=appuser
spring.datasource.password=secret123

payment.service.url=https://api-dev.example.com
logging.level.root=INFO
```

VS



ENVIRONMENT-BASED (GOOD)

Configuration in environment

```
# application.properties (no secrets/urls)
spring.datasource.url=${DB_URL}
spring.datasource.username=${DB_USER}
spring.datasource.password=${DB_PASS}

payment.service.url=${PAYMENT_URL}
logging.level.root=${LOG_LEVEL:INFO}
```

- ⚠ Different config per environment
- ⚠ Risk of exposing secrets
- ⚠ Code changes required for updates

- ✓ Same code across all environments
- ✓ Secure and portable
- ✓ Change config without redeploying code

HOW IT WORKS



Code
(Immutable)



Reads
Configuration



Runs with
Env Configuration

Example: Database URL

Local	DB_URL=jdbc:mysql://localhost:3306/app
Dev	DB_URL=jdbc:mysql://dev-db:3306/app
Test	DB_URL=jdbc:mysql://test-db:3306/app
Staging	DB_URL=jdbc:mysql://staging-db:3306/app
Prod	DB_URL=jdbc:mysql://prod-db:3306/app

COMMON SOURCES OF CONFIGURATION



Environment Variables

Simple, universal, and supported everywhere.

```
export DB_URL=...
```



Configuration Files

External files mounted or provided at runtime.

```
application.yml
```



Secrets / Config Services

Centralized management for secrets and config.

Vault, AWS Secrets Manager, etc.



Orchestrator Config

Kubernetes ConfigMaps & Secrets, etc.

```
ConfigMap, Secret
```

KEY BENEFITS



Portability

Move across environments without code changes.



Security

Keeps secrets out of source control.



Flexibility

Update configuration without rebuilding the app.



Scalability

Works seamlessly with modern cloud platforms.



Collaboration

Different teams manage code and config separately.

BEST PRACTICES

- ✓ Never commit secrets or environment-specific config.
- ✓ Use environment variables for simple key-value config.
- ✓ Use a secrets manager for credentials and sensitive data.
- ✓ Provide sensible defaults where safe, but override via env.
- ✓ Validate required config at startup and fail fast.
- ✓ Document all required configuration keys.
- ✓ Keep config consistent and versioned across environments.
- ✓ Use least-privilege access for config and secrets.



Code stays the same. Environment changes.

Keep configuration in the environment for secure, portable, and flexible applications.



BACKING SERVICES AS ATTACHED RESOURCES

Treat backing services like attached resources: easy to swap, scale, and manage.



KEY IDEA / THE PRINCIPLE

- Your app doesn't manage backing services -- it discovers and connects to them
- This enables portability, elasticity, and independence from infrastructure
- Backing services are separate from the application, provisioned and managed independently
- Connected to the app through config (environment variables)



ATTACHED RESOURCE EXAMPLES

- Database - PostgreSQL, MySQL, SQL Server, MongoDB
- Cache - Redis, Memcached
- Message Broker - Kafka, RabbitMQ, Azure Service Bus
- Object Storage - AWS S3, Azure Blob Storage, MinIO
- Search / Analytics - Elasticsearch, OpenSearch; Email / Notifications - SendGrid, SES



HOW IT WORKS

- 1. Provision Service:** a database or other service is provisioned in the platform or cloud
- 2. Attach & Configure:** connection details are provided to the application through environment variables
- 3. Application Connects:** the app reads the configuration and connects at runtime
- 4. Replace or Scale:** services can be swapped, scaled, or updated with no code changes



BAD VS GOOD APPROACH

- **Bad:** DB connection strings hard-coded in code, tight coupling, risk of leaking secrets
- **Good:** configured via environment, loose coupling, easy to swap or scale, secure and portable

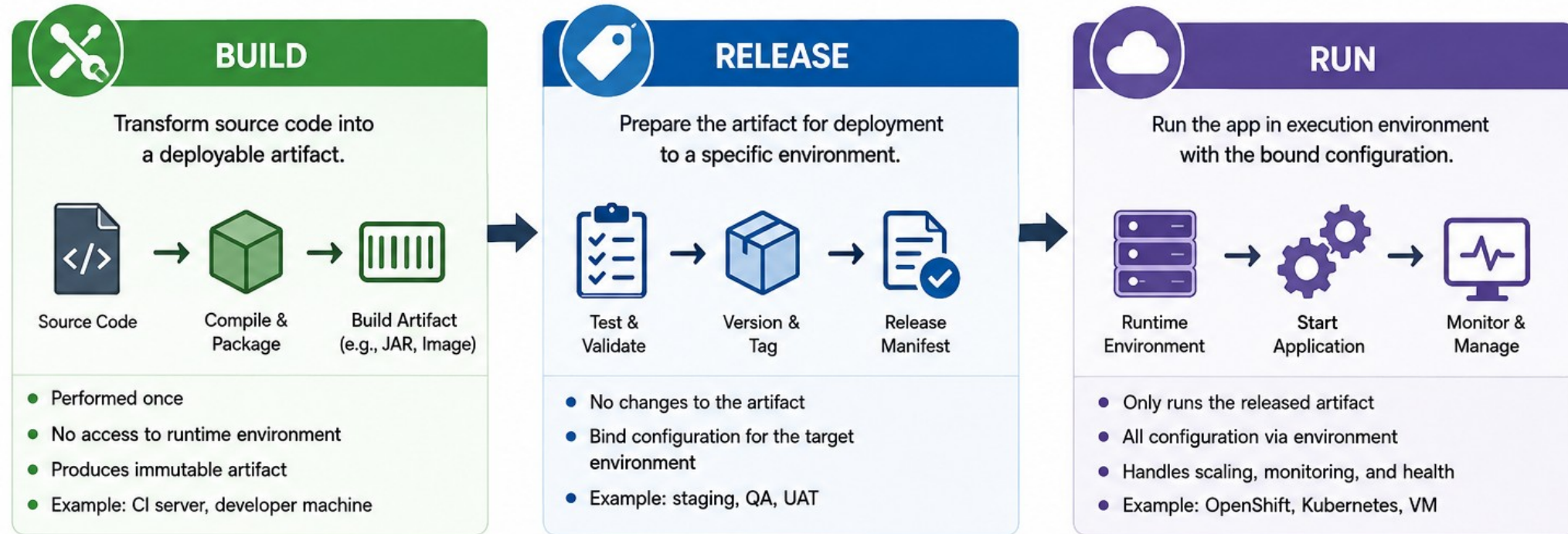
KEY TAKEAWAY Your application should be stateless and agnostic; backing services provide state and capabilities. Attach. Configure. Connect. Scale.

Separating Build, Release, and Run

Twelve-Factor App – Factor VI



Strictly separate the build, release, and run stages. Each has distinct responsibilities, different people, and different environments.



Why It Matters



Immutable Artifacts

The same artifact can be released to multiple environments without rebuilding.



Predictable Releases

Releases are repeatable, auditable, and easier to rollback.



Clear Responsibility

Different teams own build, release, and run with well-defined boundaries.



Lower Risk

Fewer changes in production reduce defects and improve operational stability.



Key Takeaways

- ✓ Build once, release many times, run everywhere.
- ✓ Never change the artifact between release and run.
- ✓ Store configuration in the environment, not in the code.
- ✓ This separation enables automation, portability, and continuous delivery.

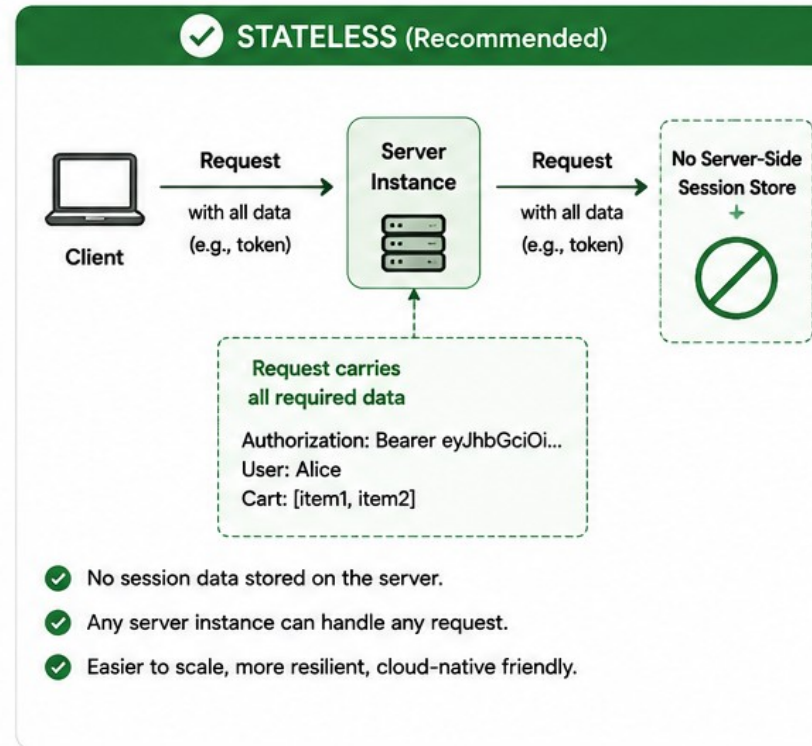
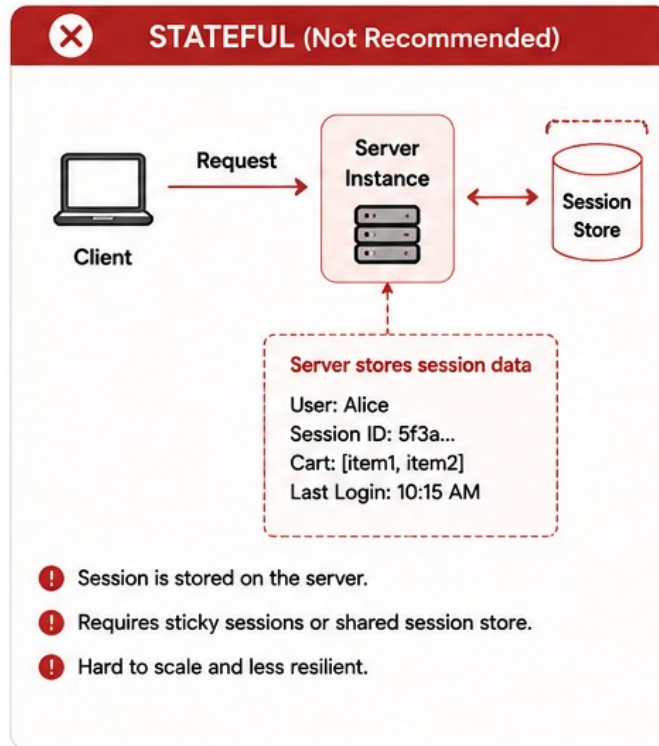
Stateless Processes

Twelve-Factor App – Factor VII



FACTOR VII

A stateless process does not store any client session information on the server.
Every request must contain all the information needed to be understood.



Benefits



Scalability

Requests can be handled by any instance—scale horizontally effortlessly.



Reliability

No single server holds critical session data. Fewer points of failure.



Deployability

Easier rolling updates and blue/green deployments.



Simplicity

Less server-side state means simpler code and operations.



Remember

Keep state in the client or external systems (e.g., databases, caches) that are explicitly attached resources.

Key Principles



Client Holds State

Store session/state on the client (e.g., in a token, cookie, or local storage).



Use Tokens

Use signed tokens (e.g., JWT) to carry identity and claims with each request.



Idempotent Requests

Design APIs so the same request can be safely retried without side effects.



Externalize State

Persist necessary data in databases, caches, or other backing services.



Example

Instead of storing a user's shopping cart in the server session, store it in Redis or send it with each request in a token or request body.

PORT BINDING AND SERVICE EXPOSURE

Services are self-contained and expose functionality by binding to a port.



KEY PRINCIPLE

- A service binds to a port and listens for incoming connections
- The port is provided by the environment
- No hard-coded ports; no reliance on external web servers
- Easier to deploy, scale, and replace



EXAMPLE (SPRING BOOT)

- Read port from environment; default to 8080 if not provided
- `@Value("${server.port:8080}") private int port;`
- Do not hard-code the port. Let the environment decide.



DEPLOYED IN CONTAINERS / PLATFORMS

- **Container -> Port Mapping (Host: 30080, Container: 8080) -> Service / Load Balancer -> Public Access**
- The platform maps external traffic to the port your service is listening on



WHY IT MATTERS

- Decoupling - the service is independent of the underlying platform or web server
- Flexibility - the same image can run in any environment with any assigned port
- Scalability - instances can be scaled horizontally without configuration changes
- Portability - works consistently across containers, VMs, and cloud platforms



BEST PRACTICES

- Bind to the port provided in the environment (e.g., `$PORT`)
- Do not assume a fixed port; expose only the required port
- Use health endpoints for readiness and liveness
- Document the service base URL and port behavior

KEY TAKEAWAY Bind to the port provided in the environment, listen for requests, and let the environment handle the rest. Goal: keep services independent, portable, and easy to scale.

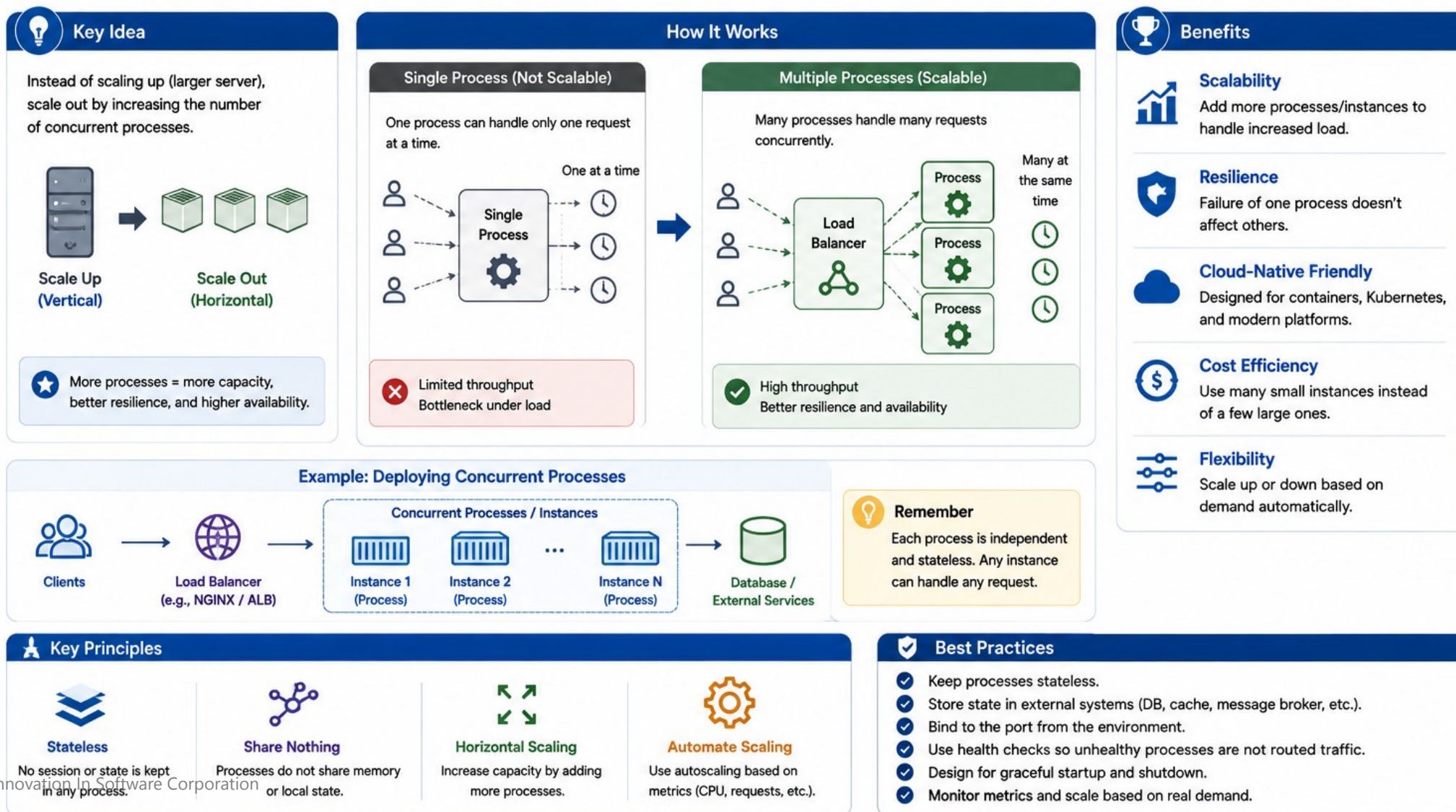
Scaling Through Process Concurrency

Twelve-Factor App – Factor IX



FACTOR IX

Scale out by running multiple instances (processes) of the application. Each process is stateless, so any instance can handle any request.



Fast Startup and Graceful Shutdown

Twelve-Factor App – Factor X

Processes start quickly and stop gracefully. Fast startup enables rapid scaling and deployment. Graceful shutdown ensures in-flight work completes and resources are released cleanly.



FACTOR X

Fast Startup

Start quickly so processes can be launched, scaled, and restarted rapidly.



Rapid scaling

New instances are ready to serve traffic in seconds.



Quick recovery

Restarted instances recover from failure fast.



Deployment agility

Blue/green, canary and rolling updates are faster and safer.



Efficient resources

Less time warming up, more time serving requests.



Better user experience

Reduced downtime and latency during changes.

Graceful Shutdown

Stop cleanly so in-flight requests finish and resources are released properly.



1. Receive Stop Signal

Process receives SIGTERM or shutdown signal.



2. Stop Accepting New Requests

Readiness set to false. Remove from load balancer.



3. Finish In-Flight Requests

Allow ongoing requests to complete within timeout.



4. Release Resources

Close connections, flush buffers, release resources.



5. Exit Cleanly

Terminate process after cleanup completes.

Spring Boot Best Practices

Fast Startup Tips



- ✓ Lazy initialization where possible.
- ✓ Avoid heavy work in constructors or @PostConstruct.
- ✓ Defer non-essential work to run asynchronously.
- ✓ Use lightweight dependencies and connections.
- ✓ Optimize classpath and reduce startup log noise.

Graceful Shutdown in Spring Boot



- ✓ Spring Boot handles SIGTERM by default.
- ✓ Set `server.shutdown=graceful` in `application.properties`.
- ✓ Configure `spring.lifecycle.timeout-per-shutdown-phase`.
- ✓ Implement `DisposableBean` or `@PreDestroy` for cleanup.
- ✓ Use `Actuator /actuator/health` readiness endpoint.



Remember

Fast startup gets you online. Graceful shutdown takes you offline—without surprising your users.

Lifecycle in a Container / Orchestrator (e.g., Kubernetes)



1. Deploy / Scale Up

New instance created



2. Fast Startup

Instance becomes ready quickly



3. Ready

Serve traffic



4. Shutdown Signal

SIGTERM received



5. Graceful Shutdown

Finish requests, clean up



6. Stopped

Process exits

Readiness = TRUE

Readiness = FALSE



Best Practices Checklist

- ✓ Applications start quickly and are ready to serve.
- ✓ Readiness probe reflects real readiness.
- ✓ On shutdown, stop accepting new traffic immediately.
- ✓ Allow in-flight requests to complete.
- ✓ Release all resources and close connections.
- ✓ Test startup and shutdown in all environments.
- ✓ Monitor startup time and shutdown duration.

DEVELOPMENT AND PRODUCTION PARITY

Keep development, staging, and production as similar as possible.



WITHOUT PARITY (COMMON PROBLEMS)

- **Development:** local machine, Local DB (H2/SQLite), different configs, manual steps, different versions
- **Production:** cloud/server, Production DB (PostgreSQL), different configs, manual steps, different versions
- Works on my machine; hard to reproduce issues; slower delivery and higher risk



KEY PRINCIPLES

- Same Codebase - use the same code and dependencies in all environments
- Same Toolchain - use the same frameworks, build tools, and infrastructure patterns
- Same Configuration - externalize configs and manage via env vars/profiles
- Same Backing Services - use the same type of databases, caches, queues, and APIs
- Automate Everything - automated builds, tests, deployments, and infrastructure as code



EXAMPLE: SPRING BOOT APPLICATION

- **Flow:** Development -> Staging -> Production
- Containerized with same Java version
- PostgreSQL in all environments; configs via environment variables
- Same CI/CD pipeline; health checks and monitoring enabled
- This parity enables smooth promotion from development to staging to production

KEY TAKEAWAY Strive for identical, or at least highly similar, environments: code, dependencies, configs, data stores, and processes.

LOGS AS EVENT STREAMS

Treat logs as event streams written to stdout/stderr, not files managed by the app.



KEY PRINCIPLE

- Logs are a stream of timestamped events describing what happens in the system
- Write them to stdout/stderr and let the environment capture, store, and process them
- Do not write to local log files; do not manage log rotation
- Do not assume log file locations -- stream logs and let the platform handle persistence



WHAT LOGS SHOULD CONTAIN

- Timestamp (preferably ISO 8601 UTC)
- Log level (TRACE, DEBUG, INFO, WARN, ERROR)
- Service / instance identity
- Correlation / trace ID
- Event message and context
- Structured data (JSON recommended)



EXAMPLE: STRUCTURED LOG (JSON)

- { **"timestamp"**: "...", "level": "INFO", "service": "order-service", "instance": "order-service-7f8d9c",
- **"traceId"**: "a1b2c3d4e5f6a7b8", "event": "OrderCreated", "orderId": "ORD-12345", "customerId": "CUST-9876", "durationMs": 124 }

KEY TAKEAWAY Logs are not just for debugging. They are a first-class stream of events that power operations, security, compliance, and insights.

LOGS AS EVENT STREAMS

Treat logs as event streams written to stdout/stderr, not files managed by the app.



END-TO-END FLOW EXAMPLE

- App Event - an event occurs in the application
- Log Emitted - log written to stdout/stderr
- Platform Captures - runtime captures the log stream
- Stored & Indexed - logs stored in central logging system
- Searched & Acted - teams query, alert, and act; correlation IDs link logs across services



COMMON ANTI-PATTERNS

- Writing logs to local files inside containers
- Relying on SSH to access log files
- Assuming log file paths or locations
- Mixing logs and user data in the same file

KEY TAKEAWAY Logs are not just for debugging. They are a first-class stream of events that power operations, security, compliance, and insights.

Administrative Processes

Twelve-Factor App – Factor XIII

Run administrative tasks (migrations, one-off scripts, diagnostics, maintenance) as one-off processes, using the same codebase and environment as regular app processes.



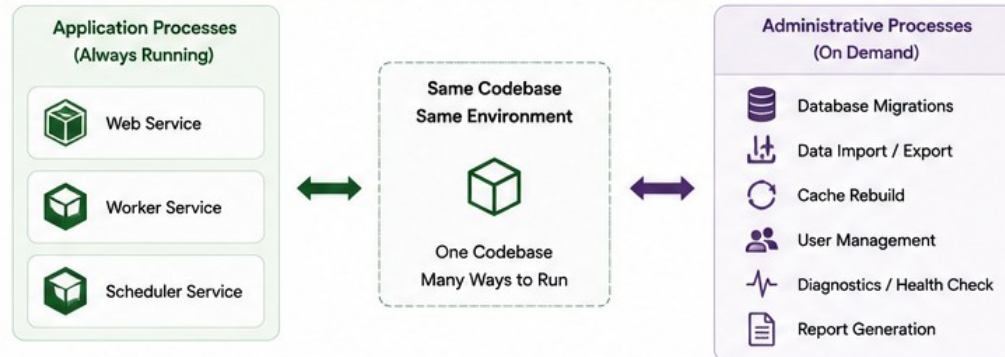
FACTOR XIII

Key Principle

Administrative tasks are executed as separate processes in the same deployed environment and codebase, not built into the main application or run manually on servers.

- ✓ Use the same codebase and configuration
- ✓ Run in the same environment as the app
- ✓ Executed on demand
- ✓ Easy to automate and audit
- ✓ Disposable and repeatable

How It Works



Administrative processes run the same released code in the same environment, ensuring consistency, security, and repeatability.

Benefits



Consistency

Uses the same code and configuration as the application.



Safety

Reduces risk of “works on my machine” issues and manual errors.



Automation

Easy to automate in CI/CD pipelines and operational tooling.



Auditability

Runs are logged, versioned, and traceable.



Resilience

Idempotent and repeatable tasks improve reliability.

Examples of Administrative Processes

- Database Migrations**
Apply schema changes using Flyway/Liquibase.
- Data Import / Export**
Load reference data or export reports.
- Cache Rebuild / Warm-up**
Preload caches for better performance.
- User / Role Management**
Create or update users, roles, or permissions.
- Diagnostics / Health Check**
Run extended diagnostics or system checks.
- Report Generation**
Generate reports or snapshots on demand.

How to Run (Examples)

Local / Dev

Run the admin command locally with the same profile and config.

```
$ java -jar app.jar admin:migrate --spring.profiles.active=dev
```

Container / Orchestrator

Run as a one-off job/pod in the same environment.

```
$ kubectl run db-migrator --image=app:1.2.0 --restart=Never -- java -jar app.jar admin:migrate
```

CI/CD Pipeline

Execute administrative tasks as a pipeline step.

```
$ name: Run DB Migrations
run: java -jar app.jar admin:migrate
```



Remember

- Administrative tasks are part of the application, not infrastructure scripts.
- They should be idempotent and safe to run multiple times.
- Run in production using the same image and configuration as the app.

Best Practices

- ✓ Keep admin tasks in the same codebase.
- ✓ Parameterize everything (no hard-coded values).
- ✓ Make tasks idempotent and safe to re-run.
- ✓ Use the same environment variables and secrets.
- ✓ Log outcomes clearly and handle failures gracefully.
- ✓ Restrict access and require proper authorization.
- ✓ Automate through CI/CD or orchestration where possible.

Anti-Patterns – Avoid



SSH into servers and run scripts manually.



Hard-coding admin logic into the main app flow.



Using different tools or configurations in production.



Creating snowflake one-off scripts.



Long-running admin tasks inside the main process.



Outcome

Administrative processes are safe, repeatable, and production-ready — executed with the same code, config, and discipline as your app.



Applying Twelve-Factor Principles to Spring Boot

Spring Boot provides first-class support for building Twelve-Factor compliant applications.

 1. Codebase Keep a single codebase in version control. Deploy the same artifact across environments.	 7. Port Binding Export services by binding to a port and listening on 0.0.0.0.
 2. Dependencies Declare and isolate dependencies explicitly. Use build tools (Maven/Gradle) for repeatable builds.	 8. Concurrency Scale out by running multiple processes. Let the platform handle concurrency.
 3. Config Store config in the environment, not in code. Use externalized configuration.	 9. Disposability Start up quickly and shut down gracefully. Processes are cheap and replaceable.
 4. Backing Services Treat backing services as attached resources via environment variables or connections.	 10. Dev/Prod Parity Keep development, staging, and production as similar as possible.
 5. Build, Release, Run Strictly separate build, release, and run stages. Produce immutable artifacts.	 11. Logs Treat logs as event streams. Send logs to stdout/stderr. Do not manage log files.
 6. Processes Execute the app as one or more stateless processes. Do not store session in memory.	 12. Admin Processes Run admin and management tasks as one-off processes using the same codebase.



Spring Boot in Action

Externalized Configuration (3. Config)

```
# application.properties
server.port=${PORT:8080}
spring.datasource.url=${DB_URL}
spring.datasource.username=${DB_USER}
spring.datasource.password=${DB_PASS}
```

Configuration via Environment Variables

```
export PORT=8080
export DB_URL=jdbc:postgresql://db:5432/orders
export DB_USER=orders_user
export DB_PASS=*****
```

Stateless Service (6. Processes)

```
@RestController
public class OrderController {
    @GetMapping("/orders/{id}")
    public Order get (@PathVariable Long id) {
        // no session or server-side state
        return service.findById(id);
    }
}
```

Logging to Stdout (11. Logs)

```
private static final Logger log =
    LoggerFactory.getLogger(OrderService.class);
log.info("Order {} processed", orderId);
```

Why It Matters



Cloud-Native Ready
Designed for portability, scalability, and resilience.



Operational Simplicity
Immutable artifacts and stateless processes simplify operations.



Team Productivity
Clear boundaries and conventions accelerate delivery.



Reliable & Secure
Consistent environments, better observability, and safer operations.



Key Takeaway

Spring Boot makes Twelve-Factor compliance straightforward through externalized config, Actuator, robust logging, and cloud-native defaults—so you can focus on delivering business value.

TWELVE-FACTOR COMPLIANCE CHECKLIST

Use this checklist to assess whether your Spring Boot microservice aligns with the Twelve-Factor App.

 COMPLIANCE CHECKLIST (1/3)

ITEM	DETAIL
1. Codebase	Is there a single codebase under version control? Use Git, one repository per service, same artifact across environments.
2. Dependencies	Are all dependencies declared explicitly? Use Maven/Gradle, avoid relying on system libraries, pin versions.
3. Config	Is configuration stored in the environment? Externalize config using application.properties/yml, env vars, or Config Server.
4. Backing Services	Are backing services attached as resources? Use environment variables or bindings (URLs, credentials); do not embed in code.

KEY TAKEAWAY Use this checklist to assess whether your Spring Boot microservice aligns with the Twelve-Factor App -- and to turn the principles into a production-readiness review.

TWELVE-FACTOR COMPLIANCE CHECKLIST

Use this checklist to assess whether your Spring Boot microservice aligns with the Twelve-Factor App.

 COMPLIANCE CHECKLIST (2/3)

ITEM	DETAIL
5. Build, Release, Run	Are build, release, and run strictly separated? Immutable artifacts; promote the same artifact through environments.
6. Processes	Is the application stateless and process-based? Do not store session in memory; store state in backing services.
7. Port Binding	Does the app bind to a port and listen on 0.0.0.0? Use server.port; do not rely on a web server.
8. Concurrency	Can the app scale out via process concurrency? Run multiple instances; no sticky sessions.
9. Disposability	Can the app start up and shut down quickly? Fast startup; handle SIGTERM for graceful shutdown.

KEY TAKEAWAY Use this checklist to assess whether your Spring Boot microservice aligns with the Twelve-Factor App -- and to turn the principles into a production-readiness review.

TWELVE-FACTOR COMPLIANCE CHECKLIST

Use this checklist to assess whether your Spring Boot microservice aligns with the Twelve-Factor App.

 COMPLIANCE CHECKLIST (3/3)

ITEM	DETAIL
10. Dev/Prod Parity	Is dev, staging, and prod as similar as possible? Same codebase, dependencies, config approach, and backing services.
11. Logs	Are logs treated as event streams? Log to stdout/stderr in structured format; do not write to local files.
12. Admin Processes	Are admin tasks run as one-off processes? Use the same codebase for admin/management tasks (e.g., migrations, data fixes).

KEY TAKEAWAY Use this checklist to assess whether your Spring Boot microservice aligns with the Twelve-Factor App -- and to turn the principles into a production-readiness review.

TWELVE-FACTOR COMPLIANCE CHECKLIST

Use this checklist to assess whether your Spring Boot microservice aligns with the Twelve-Factor App.



HOW TO USE

- Review each factor for your service
- Mark Yes, No, or N/A
- Address "No" items before production readiness
- Revisit this checklist regularly



QUICK TIPS FOR SPRING BOOT

- Externalize Everything - use environment variables, Config Server, or Secrets; never hardcode
- Immutable Artifacts - build once with Maven/Gradle; promote the same artifact
- Stateless by Design - keep no user session/state in memory; use Redis, DB, or tokens



STATUS LEGEND

- Yes - Fully compliant
- No - Not compliant
- N/A - Not applicable



WHERE THIS TAKES YOU

- **Goal:** build microservices that are portable, scalable, resilient, and easy to operate anywhere
- **Outcome:** consistent, cloud-native services that deliver business value faster

KEY TAKEAWAY Use this checklist to assess whether your Spring Boot microservice aligns with the Twelve-Factor App -- and to turn the principles into a production-readiness review.

EXERCISE 1.1 — ANALYZE A MONOLITHIC BANKING APPLICATION

Module 1 checkpoint A — Spot coupling and candidate boundaries.



OBJECTIVE

- Spot coupling and candidate boundaries
- 15 min · design checkpoint
- **Worksheet:**
curriculum/day-01/exercises/exercise-1.1-analyze-monolith.md



DO THIS

1. List major business capabilities in the monolith (profiles, accounts, transactions, loans, fraud, reporting, notifications)
2. Mark tightly coupled areas and modules that change often or carry high traffic
3. Mark where failure should be isolated so money movement is not taken down by reporting
4. Name the first extraction candidate and say why (the course extracts Account Service)



EXPECTED RESULT

- A short list of coupling problems and one justified first extract: Account Service.
- Lab 1 builds Account Service after Modules 1–3. This checkpoint is design-only.

KEY TAKEAWAY A short list of coupling problems and one justified first extract: Account Service.